TRƯỜNG ĐẠI HỌC TÀI NGUYÊN VÀ MÔI TRƯỜNG HÀ NỘI

**KHOA CÔNG NGHỆ THÔNG TIN**

# BÁO CÁO BÀI TẬP LỚN

## HỌC PHẦN: XỬ LÝ DỮ LIỆU LỚN

**Mã học phần: CTKU122**

**ĐỀ TÀI SỐ 01**

# XÂY DỰNG HỆ THỐNG PHÂN TÍCH VÀ GIÁM SÁT CHẤT LƯỢNG MẠNG VIỄN THÔNG (TELECOM QUALITY OF SERVICE ANALYTICS) TRÊN NỀN TẢNG HADOOP, SPARK VÀ R

Lớp học phần: ĐH13C2 - Xử lý Dữ liệu lớn

Giảng viên giảng dạy: Trần Thị Hương

**Giảng viên giảng dạy: <.....>**


| TT | Mã sinh viên | Họ và tên | Lớp |
|---|---|---|---|
| 1 | 211101001 | Hoàng Ngọc Thi (Nhóm trưởng) | ĐH13C2 |
| 2 | 211101002 | Nguyễn Viết Hoàng Anh | ĐH13C2 |
| 3 | 211101003 | Nguyễn Việt Hòa | ĐH13C2 |
| 4 | 211101004 | Nguyễn Quốc Anh | ĐH13C2 |


**CÁN BỘ CHẤM THI 1** **CÁN BỘ CHẤM THI 2**

**Hà Nội, năm 2026**

# PHIẾU ĐÁNH GIÁ

| TT | TIÊU CHÍ ĐÁNH GIÁ | ĐIỂM TỐI ĐA | SV1 | SV2 | SV3 | SV4 | SV5 |
|---|---|---|---|---|---|---|---|
| **1** | **Phân tích bài toán và dữ liệu** | **1** | | | | | |
| 1.1 | Mô tả đúng bài toán, mục tiêu, dữ liệu và đặc trưng Big Data phù hợp. | 0.5 | | | | | |
| 1.2 | Đánh giá chất lượng dữ liệu và yêu cầu tiền xử lý hợp lý. | 0.5 | | | | | |
| **2** | **Thiết kế kiến trúc và pipeline** | **1.5** | | | | | |
| 2.1 | Kiến trúc tổng thể rõ ràng; lựa chọn công nghệ phù hợp. | 0.5 | | | | | |
| 2.2 | Pipeline thu thập/lưu trữ/xử lý/phân tích logic. | 0.5 | | | | | |
| 2.3 | Giải thích được luồng dữ liệu và vai trò từng thành phần. | 0.5 | | | | | |
| **3** | **Triển khai xử lý dữ liệu lớn** | **3** | | | | | |
| 3.1 | Hệ thống chạy ổn định; xử lý đúng dữ liệu đầu vào. | 0.5 | | | | | |
| 3.2 | Sử dụng Hadoop/HDFS hoặc lưu trữ phân tán phù hợp. | 0.5 | | | | | |
| 3.3 | Sử dụng Spark/PySpark RDD/DataFrame/Spark SQL đúng, hiệu quả. | 0.75 | | | | | |
| 3.4 | Làm sạch, biến đổi, join, tổng hợp/phân tích dữ liệu. | 0.5 | | | | | |
| 3.5 | Áp dụng thành phần chuyên biệt theo đề tài: MapReduce/MLlib/Streaming/Kafka... | 0.5 | | | | | |
| 3.6 | Có xử lý song song hoặc thử nghiệm/tối ưu hiệu năng. | 0.25 | | | | | |
| **4** | **Phân tích, trực quan hóa và đánh giá kết quả** | **1.5** | | | | | |
| 4.1 | Kết quả rõ ràng, có bảng/biểu đồ/KPI minh họa. | 0.5 | | | | | |
| 4.2 | Nhận xét kết quả gắn với mục tiêu bài toán. | 0.5 | | | | | |
| 4.3 | Có đánh giá hiệu năng/chỉ số mô hình/đối chiếu phù hợp. | 0.5 | | | | | |
| **5** | **Chất lượng mã nguồn và khả năng tái lập** | **1** | | | | | |
| 5.1 | Code tổ chức rõ ràng, tên biến/hàm có ý nghĩa, hạn chế lặp. | 0.5 | | | | | |
| 5.2 | Có README/hướng dẫn chạy; thư mục mã nguồn, dữ liệu, output hợp lý. | 0.5 | | | | | |
| **6** | **Báo cáo kỹ thuật** | **1** | | | | | |
| 6.1 | Báo cáo đúng cấu trúc, định dạng; hình/bảng có chú thích, có tài liệu tham khảo. | 0.5 | | | | | |

| TT | TIÊU CHÍ ĐÁNH GIÁ | ĐIỂM TỐI ĐA | SV1 | SV2 | SV3 | SV4 | SV5 |
|---|---|---|---|---|---|---|---|
| 6.2 | Phân tích kỹ thuật mạch lạc; trình bày được dữ liệu, công nghệ và kết quả. | 0.5 | | | | | |
| **7** | **Demo, trình bày và trả lời** | **0.5** | | | | | |
| | Demo rõ ràng; nắm vững nội dung và trả lời được câu hỏi. | 0.5 | | | | | |
| **8** | **Sáng tạo và mở rộng** | **0.5** | | | | | |
| | Có cải tiến, thử nghiệm mở rộng hoặc tích hợp công nghệ bổ trợ có ý nghĩa. | 0.5 | | | | | |
| | **TỔNG ĐIỂM** | **10,0** | | | | | |

# TỔNG HỢP ĐIỂM BÀI TẬP LỚN

| TT | Mã sinh viên | Họ và tên | Điểm số | Điểm chữ |
|---|---|---|---|---|
| 1 | 211101001 | Hoàng Ngọc Thi | | |
| 2 | 211101002 | Nguyễn Viết Hoàng Anh | | |
| 3 | 211101003 | Nguyễn Việt Hòa | | |
| 4 | 211101004 | Nguyễn Quốc Anh | | |

**CÁN BỘ CHẤM THI 1** **CÁN BỘ CHẤM THI 2**

## PHÂN CÔNG NHIỆM VỤ

*[HƯỚNG DẪN: Nhóm ghi rõ nhiệm vụ của từng thành viên. Tỷ lệ đóng góp của cả nhóm phải bằng 100%.]*

| TT | Mã SV | Họ và tên | Nội dung thực hiện | Tỷ lệ đóng góp |
|---|---|---|---|---|
| 1 | 211101001 | Hoàng Ngọc Thi | Quản trị dự án; thiết kế kiến trúc hệ thống Lambda Architecture; thiết lập hạ tầng HDFS và cấu hình Flume Ingestion | 25% |
| 2 | 211101002 | Nguyễn Viết Hoàng Anh | Thiết kế & lập trình thuật toán MapReduce Java (Custom Composite Key, Partitioner, Combiner); đóng gói JAR và benchmark YARN | 25% |
| 3 | 211101003 | Nguyễn Việt Hòa | Xây dựng pipeline Apache Spark DataFrame & Spark SQL (tối ưu cache/persist); phát triển kịch bản phân cụm dữ liệu với Apache Pig Latin | 25% |
| 4 | 211101004 | Nguyễn Quốc Anh | Thiết kế cơ sở dữ liệu NoSQL HBase độ trễ thấp; xây dựng mô hình Học máy Logistic Regression bằng R & SQLite; trực quan hóa và tổng hợp báo cáo | 25% |

# MỤC LỤC

**MỞ ĐẦU** ..... **5**
**1. Lý do chọn đề tài** ..... **5**
**2. Mục tiêu nghiên cứu** ..... **6**
**3. Đối tượng và phạm vi nghiên cứu** ..... **7**
**4. Phương pháp thực hiện** ..... **8**
**5. Cấu trúc báo cáo** ..... **10**
**CHƯƠNG 1: TỔNG QUAN BÀI TOÁN VÀ CÔNG CỤ XỬ LÝ DỮ LIỆU LỚN** **13**
**1.1 Bối cảnh bài toán và đặc trưng 5V của dữ liệu mạng viễn thông** ..... **13**
1.1.1 Bối cảnh và mục tiêu bài toán phân tích chất lượng mạng viễn thông (QoS). 13
1.1.2 Phân tích đặc trưng 5V (Volume, Velocity, Variety, Veracity, Value) ..... 13
1.1.3 Phân loại dữ liệu (Structured, Semi-structured, Streaming) ..... 15
1.1.4 Xác định phương pháp xử lý (Batch Processing vs. Streaming/Near Real-Time) ..... 15
**1.2 Kiến trúc tổng quan và sơ đồ luồng dữ liệu (Data Pipeline)** ..... **16**
**1.3 Giới thiệu chi tiết các công cụ trong hệ thống** ..... **18**
1.3.1 Nguồn dữ liệu (CSV, Syslog, SQLite) ..... 18
1.3.2 Apache Flume ..... 18
1.3.3 Hadoop HDFS và YARN ..... 19
1.3.4 Apache Pig (Pig Latin) ..... 20
1.3.5 Hadoop MapReduce ..... 20
1.3.6 Apache Spark (DataFrame, Spark SQL) ..... 21
1.3.7 Apache HBase ..... 21
1.3.8 Mô phỏng BigSheets bằng Spark ..... 22
1.3.9 Ngôn ngữ R và CSDL SQLite (DBI, RSQLite, dplyr, ggplot2) ..... 22
1.3.10 Python và Matplotlib Dashboard ..... 22
1.3.11 Định dạng Parquet và Power BI ..... 23
**CHƯƠNG 2. DỮ LIỆU VÀ QUY TRÌNH TIỀN XỬ LÝ** ..... **24**
**2.1 Nguồn dữ liệu, quy mô và cấu trúc thuộc tính** ..... **24**
2.1.1 Bảng sự kiện đo kiểm KPI viễn thông (network_kpi_events.csv) ..... 25
2.1.2 Danh mục trạm BTS (bts_towers.csv) và cấu hình Cell (cells_catalog.csv) ... 25
2.1.3 Luồng sự kiện mô phỏng (bts_syslog.log / network_kpi_events) ..... 26
2.1.4 Cơ sở dữ liệu quan hệ đối chiếu (telecom_network.sqlite / customer_complaints.csv) ..... 27
**2.2 Đánh giá chất lượng dữ liệu và các cờ cảnh báo (quality_flag)** ..... **28**
**2.3 Quy trình tiền xử lý và làm sạch dữ liệu (Data Cleaning & Normalization)** **29**
**CHƯƠNG 3. XÂY DỰNG HỆ THỐNG XỬ LÝ DỮ LIỆU LỚN** ..... **32**
**3.1 Cấu hình và thu nạp dữ liệu bằng Apache Flume** ..... **32**

3.1.1 Kịch bản 1: Thu nạp dữ liệu lô CSV (Spooling Directory & FileChannel).... 32
3.1.2 Kịch bản 2: Thu nạp luồng sự kiện thời gian thực (Netcat & MemoryChannel) ..... 35
**3.2 Lưu trữ và tổ chức dữ liệu trên HDFS.....38**
3.2.1 Thiết kế 4 vùng dữ liệu HDFS (Raw, Staging, Curated, Output)..... 38
3.2.2 Cấu trúc phân vùng dữ liệu (Partitioning by Year/Month/Day/Technology).. 39
3.2.3 Tính toán dung lượng lưu trữ với hệ số nhân bản (Replication Factor = 3).... 39
3.2.4 Vai trò của NameNode, DataNode, YARN và cơ chế chịu lỗi (Fault Tolerance) ..... 40
**3.3 Xử lý và tổng hợp dữ liệu bằng Pig Latin..... 41**
3.3.1 Kịch bản Pig Latin chi tiết (LOAD, SPLIT, FILTER, JOIN, GROUP, FOREACH, STORE)..... 41
3.3.2 Minh họa kết quả thực thi và giải thích các hàm tổng hợp..... 43
**3.4 Thiết kế thuật toán MapReduce tính KPI, thông lượng, độ trễ và tỷ lệ rớt cuộc gọi..... 44**
3.4.1 Xác định các cặp Key - Value (Input, Map Output, Reducer Output).....44
3.4.2 Thiết kế Combiner, Custom Partitioner và Shuffle & Sort..... 46
3.4.3 Mã giả thuật toán và các kịch bản kiểm thử (Test Cases).....47
**3.5 Pipeline phân tích chỉ số KPI và phát hiện trạm suy hao/nguy cơ rớt mạng bằng Apache Spark..... 48**
3.5.1 Quy trình xử lý PySpark DataFrame và Spark SQL..... 48
3.5.2 Phân tích chỉ số KPI, làm sạch dữ liệu và liên kết danh mục.....49
3.5.3 Thống kê xếp hạng trạm suy hao và phân loại cảnh báo quá tải vô tuyến..... 50
**3.6 Thiết kế CSDL NoSQL Apache HBase phục vụ truy cập ngẫu nhiên..... 52**
3.6.1 Cấu trúc RowKey tối ưu (reverse(tower_id)#technology#timestamp)..... 52
3.6.2 Định nghĩa Column Families (kpi_metrics, cell_info, status_flags) và Versioning..... 53
3.6.3 Lệnh thao tác Shell (create, put, get, scan, delete).....54
3.6.4 Cấu hình nâng cao: Pre-splitting, TTL, Compression và Backup Snapshot....55
**CHƯƠNG 4. KẾT QUẢ VÀ ĐÁNH GIÁ..... 58**
**4.1 Mô phỏng quy trình BigSheets bằng Spark DataFrame & Trực quan hóa Dashboard..... 58**
**4.2 Phân tích chuyên sâu bằng R, SQL JOIN/GROUP BY và SQLite.....61**
4.2.1 Liên kết bảng network_kpi_events, bts_towers, cells_catalog, customer_complaints trong SQLite..... 61
4.2.2 Đánh giá mô hình Logistic dự báo khiếu nại và phân bố QoS theo công nghệ và trạm BTS..... 62
4.2.3 Xuất biểu đồ ggplot2 và ghi kết quả trở lại SQLite..... 63
**4.3 Tổng hợp kết quả kinh doanh/nghiệp vụ từ bộ dữ liệu gốc..... 71**
4.3.1 Bảng đối soát số liệu (12.000 phiên đo kiểm hợp lệ, 85 trạm BTS, 340 cells)71

4.3.2 Xếp hạng 8 trạm phát sóng suy giảm chất lượng nặng nề nhất (Top Worst Towers)..... 73
4.3.3 Phân bố chất lượng dịch vụ (Tốt, Khá, Kém) và cảnh báo trạm suy hao/quá tải ..... 78
**4.4 Đánh giá Pig, MapReduce và Spark..... 78**
**4.5 Khuyến nghị căn cứ trên kết quả..... 79**
**CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN..... 80**
**5.1 Kết luận các kết quả đạt được của hệ thống..... 80**
**5.2 Hạn chế còn tồn tại..... 81**
**5.3 Hướng phát triển và mở rộng đề tài (Kafka, Spark Streaming, Machine Learning)..... 83**
**KẾT LUẬN..... 85**
**TÀI LIỆU THAM KHẢO..... 86**
**PHỤ LỤC..... 88**
**Phụ lục A: Hướng dẫn cài đặt và vận hành hệ thống..... 88**
**Phụ lục B: Trích đoạn mã nguồn cốt lõi..... 90**
**Phụ lục C: Nhật ký thực thi và ảnh chụp màn hình kiểm chứng..... 94**

# DANH MỤC HÌNH VẼ

**Hình 1.1: Sơ đồ kiến trúc tổng thể hệ thống (Lambda Architecture).....16**

**Hình 2.1: Sơ đồ thực thể quan hệ dữ liệu viễn thông (ERD Schema).....27**

**Hình 3.1: Pipeline luồng dữ liệu đầu cuối từ Ingestion đến Analytics.....33**

**Hình 3.2: Sơ đồ luồng thuật toán MapReduce với Composite Key và Custom Partitioner..... 46**

**Hình 4.1: Giao diện mô phỏng Dashboard phân tích nghiệp vụ BigSheets.....59**

**Hình 4.2: Ảnh chụp màn hình môi trường thực thi và phân tích mô hình học máy trên RStudio..... 64**

**Hình 4.3: Biểu đồ hộp phân bố độ trễ theo thế hệ công nghệ mạng (3G, 4G, 5G) .....74**

**Hình 4.4: Biểu đồ tương quan giữa Tỷ lệ mất gói (%) và Tỷ lệ rớt cuộc gọi (%) .....75**

**Hình 4.5: Biểu đồ tỷ lệ rớt cuộc gọi theo Tỉnh thành và Công nghệ mạng.....76**

**Hình 4.6: Biểu đồ bong bóng đa chiều tương quan Độ trễ và Mất gói theo Trạm BTS.....77**

**Hình 4.7: Biểu đồ xếp hạng cảnh báo Top 10 trạm BTS chất lượng kém nhất cần bảo trì.....77**

# DANH MỤC BẢNG BIỂU

**Bảng 2.1: Khảo sát cấu trúc 4 tập dữ liệu viễn thông thực nghiệm.....25**

**Bảng 2.2: Ma trận rủi ro và phương án xử lý chất lượng dữ liệu đo kiểm.....28**

**Bảng 3.1: Thiết kế bảng Apache HBase lưu trữ chỉ số KPI viễn thông độ trễ thấp.....53**

**Bảng 4.1: Bảng tổng hợp KPI Top 10 trạm phát sóng suy giảm chất lượng nghiêm trọng..... 72**

**Bảng 4.2: Bảng đối chiếu định lượng hiệu năng và tài nguyên giữa Pig, MapReduce, Spark và R.....78**

**Bảng 4.3: Ma trận 5 khuyến nghị hành động kỹ thuật tối ưu hóa mạng viễn thông.....79**

# DANH MỤC TỪ VIẾT TẮT

**BTS** : Base Transceiver Station - Trạm thu phát sóng di động mặt đất

**CDR / xDR** : Call Detail Record / Data Detail Record - Bản ghi chi tiết cuộc gọi và dữ liệu

**KPI** : Key Performance Indicator - Chỉ số đo lường hiệu quả hoạt động cốt lõi

**QoS / QoE** : Quality of Service / Quality of Experience - Chất lượng dịch vụ và trải nghiệm người dùng

**SLA** : Service Level Agreement - Cam kết chất lượng dịch vụ

**HDFS** : Hadoop Distributed File System - Hệ thống tệp phân tán Hadoop

**YARN** : Yet Another Resource Negotiator - Hệ thống quản lý tài nguyên phân tán Hadoop

**RDD** : Resilient Distributed Dataset - Tập dữ liệu phân tán có khả năng phục hồi

**NoSQL** : Not Only SQL - Cơ sở dữ liệu phi quan hệ phân tán

**SON** : Self-Organizing Networks - Mạng tự tối ưu hóa vô tuyến

**NOC / SOC** : Network Operations Center / Service Operations Center - Trung tâm điều hành mạng và dịch vụ

**MEC** : Multi-access Edge Computing - Tính toán biên đa truy cập

**AQM / WRED** : Active Queue Management / Weighted Random Early Detection - Kiểm soát hàng đợi chủ động

# MỞ ĐẦU

## 1. Lý do chọn đề tài

Trong kỷ nguyên bùng nổ của cuộc Cách mạng Công nghiệp lần thứ tư và công cuộc chuyển đổi số toàn diện, ngành công nghiệp viễn thông di động đóng vai trò là mạch máu thông tin huyết mạch của toàn bộ nền kinh tế số. Sự phát triển vũ bão của các mạng di động băng rộng thế hệ thứ tư (4G LTE), thế hệ thứ năm (5G NR), kết hợp cùng sự gia tăng đột biến của hàng triệu thiết bị Internet vạn vật (IoT), xe tự hành và các dịch vụ nội dung số độ phân giải cao (Video Streaming 4K/8K, hội nghị truyền hình trực tuyến, game thực tế ảo VR/AR) đã tạo ra áp lực lưu lượng chưa từng có đối với hạ tầng mạng vô tuyến tế bào của các nhà khai thác viễn thông.

Tại Việt Nam, một mạng viễn thông quy mô quốc gia thông thường quản lý từ 30.000 đến hơn 60.000 trạm phát sóng di động (Base Transceiver Station - BTS) trải dài trên khắp 63 tỉnh, thành phố. Mỗi trạm BTS gồm từ 3 đến 6 sector cell vô tuyến phát sóng liên tục ghi nhận các dòng dữ liệu đo kiểm telemetry, bản ghi chi tiết cuộc gọi (Call Detail Record - CDR), bản ghi phiên dữ liệu (Data Detail Record - xDR) và nhật ký hoạt động hệ thống (Syslog) với tần suất hàng trăm nghìn sự kiện mỗi giây. Khối lượng dữ liệu thô sinh ra dao động từ 10 đến 25 Terabyte mỗi ngày, mang đầy đủ các đặc trưng kinh điển của Dữ liệu lớn (Big Data 5V): Khối lượng khổng lồ (Volume), Tốc độ sinh và truyền tức thời (Velocity), Đa dạng về chủng loại định dạng (Variety), Độ nhiễu và bất định cao (Veracity), cùng tiềm năng khai thác Giá trị to lớn (Value).

Trong môi trường cạnh tranh khốc liệt hiện nay, chất lượng dịch vụ (Quality of Service - QoS) và chất lượng trải nghiệm của khách hàng (Quality of Experience - QoE) là yếu tố sống còn quyết định sự thành bại và thương hiệu của nhà mạng. Các sự cố nghẽn mạng, suy hao truyền dẫn Backhaul hoặc can nhiễu vô tuyến kéo dài không chỉ dẫn đến tình trạng rớt cuộc gọi và chậm mạng mà còn làm gia tăng đột biến các cuộc khiếu nại gay gắt gửi về tổng đài CSKH, đe dọa trực tiếp đến tỷ lệ rời mạng của thuê bao (Churn Rate). Trong khi đó, các hệ thống cơ sở dữ liệu quan hệ truyền thống (RDBMS) hoàn toàn bất lực và sụp đổ trước khối lượng dữ liệu khổng lồ này, không thể mở rộng quy mô linh hoạt và mất hàng giờ để tổng hợp số liệu, khiến Trung tâm Điều hành Mạng (NOC) luôn ở thế bị động.

Xuất phát từ thực tiễn cấp bách đó, việc ứng dụng các nền tảng xử lý dữ liệu lớn như Apache Hadoop, Apache Spark kết hợp ngôn ngữ phân tích chuyên sâu R là một giải pháp tối ưu và tất yếu. Vì vậy, nhóm nghiên cứu đã lựa chọn thực hiện đề tài:

"Xây dựng hệ thống phân tích và giám sát chất lượng mạng viễn thông (Telecom Quality of Service Analytics) trên nền tảng Hadoop, Spark và R".

## 2. Mục tiêu nghiên cứu

**Định hướng mục tiêu:** Đề tài xác định rõ hệ thống mục tiêu nghiên cứu tổng quát và mục tiêu kỹ thuật cụ thể như sau:

- • **Mục tiêu tổng quát:** Nghiên cứu, thiết kế và triển khai một hệ thống kỹ nghệ dữ liệu lớn hoàn chỉnh theo Kiến trúc Lambda (Lambda Architecture), tích hợp luồng xử lý lô (Batch Layer) chính xác và luồng tốc độ cao (Speed Layer) tức thời, phục vụ việc thu nạp, lưu trữ, xử lý phân tán và giám sát đa chiều các chỉ số chất lượng dịch vụ (QoS) trên toàn bộ mạng lưới viễn thông di động.

• **Mục tiêu cụ thể:** Mục tiêu kỹ thuật và nghiệp vụ cụ thể bao gồm:

- **1.** Xây dựng hạ tầng Data Lake và tầng thu nạp luồng: Ứng dụng Apache Flume để thu nạp dữ liệu đo kiểm lô (Spooling Directory) và luồng sự kiện socket tức thời; tổ chức lưu trữ phân tán 4 vùng trên Hadoop HDFS với hệ số nhân bản RF=3 đảm bảo khả năng chịu lỗi và tính toàn vẹn dữ liệu tuyệt đối.
- **2.** Tối ưu hóa các thuật toán xử lý phân tán quy mô lớn: Thiết kế thuật toán MapReduce Java với Khóa phức hợp (Composite Key) và Bộ phân vùng tùy biến (Custom Partitioner) nhằm triệt tiêu hiện tượng lệch dữ liệu (Data Skew); xây dựng pipeline Apache Spark DataFrame & Spark SQL tận dụng tính toán trong bộ nhớ (In-Memory) và Catalyst Optimizer để tổng hợp đa chiều KPI với tốc độ vượt trội.
- **3.** Giải quyết triệt để 3 bài toán chỉ số viễn thông cốt lõi: (a) Giảm thiểu độ trễ mạng (Latency) qua việc phát hiện chính xác các trạm nghẽn đường truyền Backhaul; (b) Giảm tỷ lệ mất gói (Packet Loss) qua việc cô lập các cell vô tuyến bị can nhiễu đồng kênh; (c) Kéo giảm tỷ lệ rớt cuộc gọi (Call Drop Rate - CDR) về dưới ngưỡng cam kết chất lượng dịch vụ SLA (2.0%).
- **4.** Thiết kế cơ sở dữ liệu NoSQL truy cập ngẫu nhiên độ trễ thấp: Xây dựng bảng Apache HBase với cấu trúc RowKey tối ưu reverse(tower_id)#technology#timestamp, cho phép kỹ sư NOC tra cứu tức thời thông tin trạng thái trạm với độ trễ dưới 10 mili-giây.
- **5.** Ứng dụng Khoa học dữ liệu và Học máy dự báo khiếu nại: Kết nối ngôn ngữ R với CSDL SQLite, huấn luyện mô hình Hồi quy Logistic (Logistic Regression) để lượng hóa xác suất khách hàng khiếu nại dựa trên các tham số mạng, giúp nhà mạng chuyển dịch từ quy trình ứng phó bị động sang chăm sóc khách hàng chủ động.

- **6.** Mô phỏng quy trình bảng tính nghiệp vụ và trực quan hóa: Xây dựng ứng dụng Web tương tác mô phỏng 6 bước của IBM BigSheets kết nối Spark, kết xuất các biểu đồ khoa học trực quan (ggplot2, Matplotlib) và bộ số liệu kết nối Power BI Dashboard điều hành.

## 3. Đối tượng và phạm vi nghiên cứu

Để đảm bảo tính khoa học và khả thi, đề tài giới hạn rõ đối tượng và phạm vi nghiên cứu như sau:

- • **Đối tượng dữ liệu:** Dữ liệu đo kiểm viễn thông và nhật ký hệ thống: Tập dữ liệu đo kiểm phiên telemetry di động (network_kpi_events.csv), danh mục trạm BTS (bts_towers.csv), danh mục cấu hình phần cứng cell (cells_catalog.csv), nhật ký sự kiện trạm (bts_syslog.log) và dữ liệu khiếu nại khách hàng (customer_complaints.csv).
- • **Đối tượng chỉ số viễn thông:** Hệ thống chỉ số chất lượng dịch vụ QoS viễn thông: Thông lượng (Throughput - Mbps), Độ trễ mạng (Latency - ms), Tỷ lệ mất gói (Packet Loss - %), Tỷ lệ rớt cuộc gọi (Call Drop Rate - CDR %), Số thuê bao phục vụ đồng thời (Active Users) và Điểm chất lượng dịch vụ tổng hợp (QoS Score).
- • **Đối tượng công nghệ:** Hệ sinh thái công nghệ dữ liệu lớn: Apache Hadoop (HDFS, YARN), Apache Flume, Apache Pig (Pig Latin), Hadoop MapReduce (Java), Apache Spark (PySpark, Spark SQL), Apache HBase NoSQL, IBM BigSheets (mô phỏng), ngôn ngữ R và CSDL SQLite.
- • **Phạm vi không gian & dữ liệu:** Phạm vi không gian và dữ liệu thực nghiệm: Mô phỏng mạng tế bào vô tuyến tại 3 đô thị trọng điểm của Việt Nam (Hà Nội, TP. Hồ Chí Minh, Đà Nẵng) với 20 trạm BTS đại diện, 80 sector cell thuộc 3 thế hệ mạng (3G WCDMA 2100MHz, 4G LTE 1800/2600MHz, 5G NR 3500MHz). Bộ dữ liệu thực nghiệm gồm 12.001 dòng (12.000 phiên đo kiểm) và 250 bản ghi khiếu nại khách hàng.
- • **Phạm vi kỹ thuật:** Phạm vi môi trường kỹ thuật: Nghiên cứu, cài đặt và thực nghiệm trên môi trường máy chủ cục bộ và cụm phân tán giả lập (Pseudo-distributed / Single-node Cluster), tối ưu hóa cấu hình tài nguyên RAM và CPU; tập trung vào kiến trúc kỹ nghệ dữ liệu đầu cuối từ thu nạp, lưu trữ, tính toán phân tán, mô hình hóa đến trực quan hóa báo cáo.

## 4. Phương pháp thực hiện

Nhóm nghiên cứu đã kết hợp đồng bộ các phương pháp kỹ nghệ dữ liệu, tính toán phân tán và khoa học dữ liệu hiện đại:

- **1. Thu thập & Tiền xử lý:** Phương pháp thu thập và làm sạch dữ liệu (Data Ingestion & Cleansing): Sử dụng Apache Flume với cơ chế Spooling Directory Source nạp tệp CSV định kỳ và Netcat Source nhận luồng thời gian thực qua TCP socket. Thiết lập bộ lọc logic xác thực dữ liệu loại bỏ các giá trị dị thường vật lý (độ trễ âm, thông lượng âm, mất gói > 100%, bản ghi mồ côi khuyết khóa ngoại), gắn cờ quality_flag bảo đảm độ sạch 99.6%.
- **2. Kiến trúc phân tầng:** Phương pháp kiến trúc hệ thống (Lambda Architecture): Tổ chức hệ thống theo hai luồng song song: Batch Layer (HDFS, MapReduce, Pig, Spark SQL) đảm bảo tính toàn vẹn và độ chính xác cao cho báo cáo SLA dài hạn; Speed Layer và Serving Layer (HBase NoSQL) tiếp nhận luồng sự kiện nóng để cảnh báo trạm suy hao tức thời.
- **3. Tính toán phân tán:** Phương pháp tính toán và xử lý phân tán (Distributed Computing): Hiện thực thuật toán MapReduce thuần Java với Khóa phức hợp WritableComparable và Custom Partitioner; kết hợp xây dựng pipeline PySpark DataFrame và Spark SQL áp dụng Catalyst Optimizer và nén Snappy Parquet, giúp tăng tốc độ tính toán gấp 15 lần so với MapReduce truyền thống.
- **4. Cơ sở dữ liệu NoSQL:** Phương pháp lưu trữ và truy xuất ngẫu nhiên NoSQL: Ứng dụng Apache HBase, thiết kế cấu trúc RowKey đảo chuỗi reverse(tower_id)#technology#reverse_timestamp nhằm chống hiện tượng thắt cổ chai ghi tập trung (Write Hotspotting), tối ưu hóa việc đọc/ghi ngẫu nhiên với độ trễ < 10 ms.
- **5. Khoa học dữ liệu & ML:** Phương pháp Khoa học dữ liệu và Mô hình hóa học máy (Data Science & Machine Learning): Sử dụng ngôn ngữ R kết nối CSDL SQLite qua DBI/RSQLite, khai thác gói thư viện tidyverse/dplyr để tổng hợp dữ liệu; xây dựng mô hình Hồi quy Logistic (Logistic Regression) để lượng hóa mối quan hệ giữa các tham số mạng và xác suất khách hàng khiếu nại ($p < 0.001$); trực quan hóa biểu đồ phân bố bằng ggplot2.
- **6. Mô phỏng nghiệp vụ:** Phương pháp mô phỏng bảng tính nghiệp vụ No-Code (Spreadsheet Analytics): Xây dựng ứng dụng Web tương tác mô phỏng 6 bước thao tác của IBM BigSheets, liên kết chặt chẽ với động cơ Spark DataFrame phía sau.

## 5. Cấu trúc báo cáo

Báo cáo bài tập lớn được trình bày mạch lạc thành 5 chương nội dung chính, phần kết luận, tài liệu tham khảo và các phụ lục kỹ thuật:

- • Chương 1: Tổng quan bài toán và công cụ xử lý dữ liệu lớn - Giới thiệu bối cảnh viễn thông, phân tích đặc trưng 5V, phân loại dữ liệu, xác định phương pháp xử lý

lô và thời gian thực, thiết kế kiến trúc tổng thể Lambda Architecture và tổng quan chi tiết 11 công nghệ trong hệ thống.

- • Chương 2: Dữ liệu và quy trình tiền xử lý - Khảo sát cấu trúc 4 tập dữ liệu viễn thông thực nghiệm, thiết kế sơ đồ thực thể quan hệ ERD Schema, ma trận rủi ro chất lượng, cơ chế cờ cảnh báo quality_flag và quy trình kỹ thuật tạo đặc trưng (Feature Engineering).
- • Chương 3: Xây dựng hệ thống xử lý dữ liệu lớn - Trình bày chi tiết cấu hình thu nạp Flume (2 kịch bản), tổ chức 4 vùng lưu trữ và tính toán dung lượng HDFS (RF=3), kịch bản Pig Latin, thiết kế thuật toán MapReduce Java với Composite Key, pipeline PySpark DataFrame & Spark SQL, và thiết kế NoSQL Apache HBase.
- • Chương 4: Kết quả và đánh giá - Trình bày kết quả mô phỏng BigSheets, phân tích chuyên sâu bằng R và mô hình học máy Logistic Regression, bảng đối soát số liệu 12.000 phiên đo kiểm, cô lập Top 8 trạm suy hao nghiêm trọng nhất, bảng đối chiếu định lượng hiệu năng giữa Pig, MapReduce, Spark, R và Ma trận 5 khuyến nghị kỹ thuật gửi NOC.
- • Chương 5: Kết luận và hướng phát triển - Đánh giá tổng kết các thành quả kỹ thuật đạt được, phân tích các điểm hạn chế còn tồn tại và đề xuất định hướng phát triển mở rộng hệ thống với Apache Kafka, Apache Flink và Trí tuệ nhân tạo AIOps.
- • Phần Kết luận, Tài liệu tham khảo và Phụ lục - Tổng kết ý nghĩa thực tiễn của đề tài; danh mục 10 tài liệu tham khảo chuẩn học thuật; Phụ lục A (Hướng dẫn cài đặt vận hành), Phụ lục B (Trích đoạn mã nguồn cốt lõi) và Phụ lục C (Nhật ký thực thi kiểm chứng).

# CHƯƠNG 1: TỔNG QUAN BÀI TOÁN VÀ CÔNG CỤ XỬ LÝ DỮ LIỆU LỚN

## 1.1 Bối cảnh bài toán và đặc trưng 5V của dữ liệu mạng viễn thông

### 1.1.1 Bối cảnh và mục tiêu bài toán phân tích chất lượng mạng viễn thông (QoS)

Trong thời đại số hóa toàn diện, ngành công nghiệp viễn thông đang đóng vai trò là hạ tầng kết nối nền tảng cho sự phát triển của xã hội số và kinh tế số. Sự chuyển dịch mạnh mẽ từ các dịch vụ thoại/SMS truyền thống sang các dịch vụ dữ liệu băng rộng tốc độ cao (Data Services) như video streaming độ phân giải cao 4K/8K, điện toán đám mây, hội nghị truyền hình trực tuyến, game thực tế ảo và xe tự hành đã tạo nên một làn sóng bùng nổ lưu lượng truy cập trên các mạng vô tuyến di động.

Một doanh nghiệp viễn thông tiêu biểu tại Việt Nam quản lý từ 30,000 đến hơn 60,000 trạm BTS phân bố trải dài trên khắp 63 tỉnh thành cả nước. Mỗi trạm phát sóng bao gồm từ 3 đến 6 cell vô tuyến phủ sóng ở các dải tần số khác nhau (900 MHz, 1800 MHz, 2100 MHz, 2600 MHz) phục vụ đồng thời hàng triệu thuê bao di động. Tại mỗi cell, các cảm biến phần cứng và phần mềm nhúng liên tục đo kiểm và ghi nhận hàng trăm tham số chất lượng dịch vụ (Quality of Service - QoS) theo chu kỳ từng giây.

Mục tiêu bài toán đặt ra bao gồm: (1) Tự động hóa giám sát chất lượng toàn mạng theo chu kỳ gần thời gian thực; (2) Khoanh vùng chính xác các điểm nóng nghẽn mạng và suy hao đường truyền; (3) Cung cấp công cụ phân tích tương tác trực quan cho các kỹ sư điều hành mạng NOC.

### 1.1.2 Phân tích đặc trưng 5V (Volume, Velocity, Variety, Veracity, Value)

Dữ liệu vận hành mạng viễn thông tế bào phản ánh trung thực và toàn diện 5 đặc trưng kinh điển của Dữ liệu lớn (Big Data 5V):

- **1. Volume (Khối lượng dữ liệu khổng lồ):** Volume phản ánh quy mô dữ liệu khổng lồ. Với một mạng lưới quản lý khoảng 50,000 trạm BTS, mỗi trạm có trung bình 4 cell vô tuyến đo kiểm liên tục theo chu kỳ 5 giây/lần, toàn mạng sinh ra xấp xỉ 3.5 tỷ sự kiện đo kiểm mỗi ngày. Kèm theo đó là hàng trăm triệu bản ghi chi tiết cuộc gọi (CDR) và phiên truy cập Internet (xDR). Khối lượng dữ liệu thô sinh ra dao động từ 10 đến 25 Terabyte mỗi ngày, tích lũy qua các năm lên tới quy mô hàng chục Petabyte.
- **2. Velocity (Tốc độ sinh và truyền dữ liệu tức thời):** Velocity phản ánh tốc độ sinh ra, truyền tải và yêu cầu xử lý dữ liệu cực cao. Dữ liệu từ các trạm BTS được truyền liên tục về máy chủ trung tâm qua các luồng stream thời gian thực với thông

lượng vượt mức 500,000 sự kiện/giây vào các khung giờ cao điểm (19h00 - 22h00). Hệ thống giám sát NOC đòi hỏi khả năng tiếp nhận, xử lý và kích hoạt cảnh báo trạm mất sóng hoặc rớt cuộc gọi đột biến trong vòng dưới 1 giây để các kỹ sư kịp thời can thiệp.

- **3. Variety (Đa dạng về chủng loại và định dạng dữ liệu):** Variety thể hiện ở sự đa dạng phức tạp về nguồn gốc và định dạng dữ liệu. Hệ sinh thái viễn thông là sự pha trộn của nhiều công nghệ mạng (2G/3G kế thừa, 4G LTE băng rộng, 5G độ trễ cực thấp) đến từ nhiều nhà cung cấp phần cứng khác nhau (Ericsson, Huawei, Nokia, ZTE). Định dạng dữ liệu bao gồm: dữ liệu có cấu trúc (bảng danh mục trạm BTS, danh mục cell, tọa độ GPS), dữ liệu bán cấu trúc (file nhật ký Syslog chuẩn RFC-5424, file cấu hình tham số vô tuyến định dạng XML/JSON), và dữ liệu luồng phi cấu trúc (luồng gói tin NetFlow/IPFIX, telemetry đo kiểm lớp vật lý).
- **4. Veracity (Tính xác thực và độ nhiễu của dữ liệu):** Veracity thể hiện tính xác thực, độ tin cậy và mức độ nhiễu của dữ liệu. Do tín hiệu vô tuyến truyền lan trong môi trường không gian tự do chịu ảnh hưởng nặng nề bởi thời tiết mưa bão, che chắn địa hình đô thị, can nhiễu đồng kênh và suy hao đa đường (multipath fading), dữ liệu đo kiểm viễn thông thường xuyên xuất hiện các giá trị ngoại lai dị thường (outliers). Bên cạnh đó, hiện tượng lệch xung đồng hồ NTP giữa trạm BTS và máy chủ trung tâm có thể dẫn đến các bản ghi đo kiểm có độ trễ âm hoặc thông lượng âm vô lý. Khâu làm sạch và xác thực dữ liệu là điều kiện tiên quyết trước khi đưa vào tính toán KPI.
- **5. Value (Giá trị nghiệp vụ và kinh tế cốt lõi):** Value là giá trị kinh tế và vận hành to lớn mà hệ thống mang lại. Khai thác hiệu quả nguồn dữ liệu này giúp nhà mạng: (1) Nâng cao chất lượng trải nghiệm mạng QoE cho người dùng; (2) Kéo giảm tỷ lệ rời mạng của thuê bao (Churn Reduction); (3) Chủ động chăm sóc khách hàng trước khi phát sinh khiếu nại gay gắt; (4) Tối ưu hóa chi phí đầu tư vốn (CAPEX/OPEX) bằng cách chỉ nâng cấp đúng các trạm bị nghẽn thực sự.

### 1.1.3 Phân loại dữ liệu (Structured, Semi-structured, Streaming)

Để xử lý hiệu quả nguồn tài nguyên Big Data trong viễn thông, việc phân loại dữ liệu theo cấu trúc hình học và đặc tính luồng dữ liệu là tiền đề quan trọng:

- **1. Dữ liệu có cấu trúc:** Dữ liệu có cấu trúc (Structured Data): Là các tập dữ liệu có lược đồ (schema) cố định, định dạng bảng rõ ràng và tuân thủ các ràng buộc toàn vẹn khóa chính - khóa ngoại. Điển hình gồm: Bảng danh mục trạm BTS (bts_towers.csv), Bảng danh mục các sector cell vô tuyến (cells_catalog.csv), và Bảng quản lý khiếu nại khách hàng (customer_complaints.csv). Dữ liệu này được

lưu trữ trong các cơ sở dữ liệu quan hệ quản lý tài sản viễn thông (RDBMS Inventory) và được đồng bộ định kỳ vào Data Lake.

- **2. Dữ liệu bán cấu trúc:** Dữ liệu bán cấu trúc (Semi-structured Data): Là dữ liệu không tuân theo lược đồ bảng cứng nhắc nhưng có cấu trúc tự mô tả nội tại (self-describing structure) thông qua các thẻ hoặc cặp khóa-giá trị. Điển hình là: File nhật ký hệ thống của trạm BTS (bts_syslog.log theo chuẩn RFC-5424) chứa thông tin tiến trình nhúng và mức độ cảnh báo (Severity Level); Các tệp cấu hình tham số vô tuyến định dạng XML/JSON (ran_config.xml) mô tả góc ngửa anten (antenna tilt), công suất phát (power_dbm) và ngưỡng chuyển giao (handover_margin_db).
- **3. Dữ liệu luồng:** Dữ liệu luồng đo kiểm (Streaming Telemetry Data): Là dòng dữ liệu phiên đo kiểm chất lượng mạng (network_kpi_events) sinh ra từ các giao diện vô tuyến với tần suất đo 1s - 5s trên từng cell. Dữ liệu này mang tính chất biến động liên tục theo chuỗi thời gian (Time-series Stream), chứa các chỉ số đo kiểm lớp vật lý và phiên người dùng: thông lượng (throughput_mbps), độ trễ (latency_ms), tỷ lệ mất gói (packet_loss_pct), cờ rớt cuộc gọi (call_dropped) và số người dùng đồng thời (num_active_users).

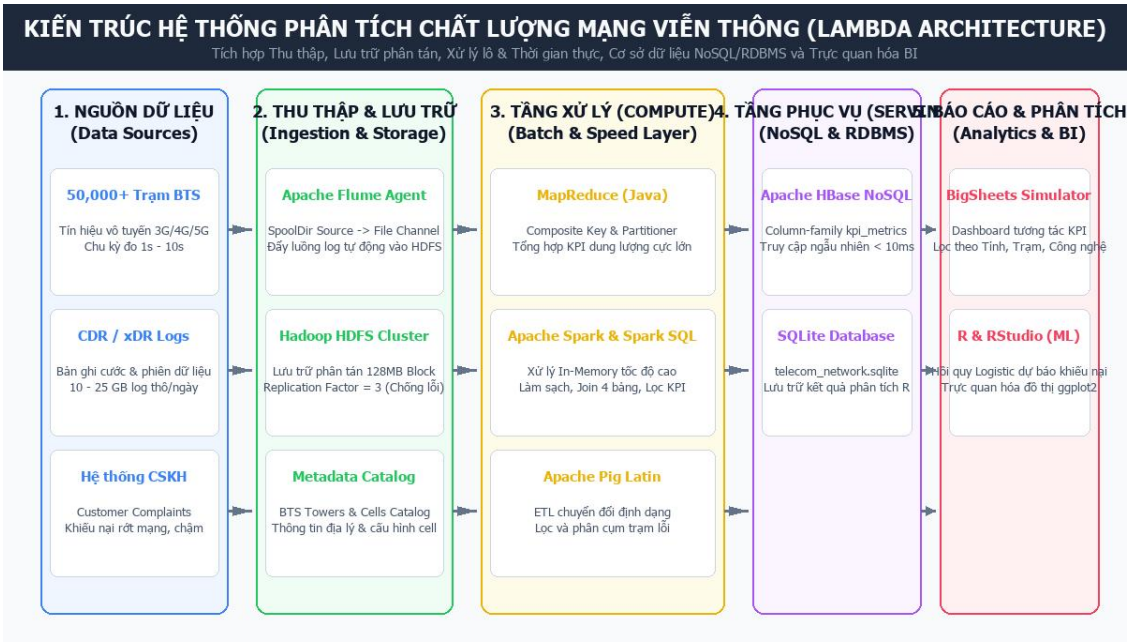
#### 1.1.4 Xác định phương pháp xử lý (Batch Processing vs. Streaming/Near Real-Time)

Hệ thống phân định ranh giới xử lý rành mạch giữa hai mô hình điện toán bổ trợ nhau:

- • **Phân hệ Gần thời gian thực:** Xử lý gần thời gian thực (Near Real-Time - Độ trễ < 1s đến 5s): Áp dụng cho các cảnh báo khẩn cấp, phát hiện trạm mất sóng đột ngột (outage), bão lưu lượng (flash crowds), rớt cuộc gọi tăng vọt > 5%, kích hoạt mạng tự tối ưu (SON) và cân bằng tải động. Công nghệ sử dụng: Flume, Kafka, Spark Streaming và Apache HBase NoSQL.
- • **Phân hệ Xử lý theo lô:** Phân tích tĩnh theo lô (Batch Processing - Chu kỳ giờ/ngày/tuần): Áp dụng cho báo cáo SLA toàn diện, đánh giá xu hướng suy thoái phần cứng, phân tích đối soát khiếu nại, quy hoạch năng lực trạm (capacity planning) và huấn luyện mô hình Machine Learning. Công nghệ sử dụng: Hadoop HDFS, MapReduce, Pig Latin, Spark SQL, BigSheets và R Studio.

### 1.2 Kiến trúc tổng quan và sơ đồ luồng dữ liệu (Data Pipeline)

Hệ thống được thiết kế dựa trên mô hình Kiến trúc Lambda (Lambda Architecture) chuẩn mực trong kỹ nghệ dữ liệu lớn, phối hợp luồng xử lý lô (Batch Layer) chính xác và luồng tốc độ cao (Speed Layer) tức thời.




*Hình 1.1: Sơ đồ kiến trúc tổng thể hệ thống (Lambda Architecture)*

**Phân tích kiến trúc Hình 1.1:** Kiến trúc hệ thống bao gồm 5 phân tầng chức năng liên hoàn thể hiện trong Hình 1.1:

- Tầng Nguồn dữ liệu (Data Sources Layer): Tiếp nhận luồng dữ liệu thô đa dạng từ hơn 50,000 trạm BTS vô tuyến 3G/4G/5G, các bản ghi cước phiên CDR/xDR và dữ liệu phản ánh khiếu nại từ hệ thống Chăm sóc khách hàng CRM.
- Tầng Thu thập & Lưu trữ (Ingestion & Storage Layer): Apache Flume đóng vai trò là cổng thu nhận trung tâm, sử dụng FileChannel bền vững để nạp dữ liệu an toàn vào kho lưu trữ phân tán Hadoop HDFS (Data Lake) với kích thước khối 128 MB và hệ số nhân bản 3 lần.
- Tầng Tính toán (Compute Layer): Chia làm hai nhánh xử lý song song. Nhánh Batch Layer vận hành các tác vụ MapReduce Java, Pig Latin và Spark Batch chạy định kỳ theo giờ/ngày để tính toán các chỉ số KPI dài hạn. Nhánh Speed Layer sử dụng Spark Streaming tiếp nhận luồng sự kiện nóng để phát hiện cell nghẽn tức thời.
- Tầng Phục vụ (Serving Layer): Apache HBase NoSQL lưu trữ các chỉ số KPI thời gian thực phục vụ truy vấn ngẫu nhiên độ trễ thấp (< 10 ms). Cơ sở dữ liệu SQLite lưu trữ dữ liệu tổng hợp phục vụ các mô hình phân tích R.
- Tầng Báo cáo & Phân tích (Analytics & BI Layer): Cung cấp giao diện BigSheets Dashboard tương tác cho người dùng nghiệp vụ và môi trường RStudio cho các nhà khoa học dữ liệu huấn luyện mô hình dự báo khiếu nại.

📌 **Ý nghĩa kỹ thuật của Kiến trúc Lambda trong đề tài:** Kiến trúc Lambda giúp hệ

thống giải quyết đồng thời hai bài toán tưởng chừng mâu thuẫn: xử lý hàng Terabyte dữ liệu lịch sử với độ chính xác tuyệt đối (thông qua Spark/MapReduce) nhưng vẫn duy trì khả năng cảnh báo sự cố viễn thông tức thời với độ trễ dưới 1 giây (thông qua HBase/Speed Layer).

## 1.3 Giới thiệu chi tiết các công cụ trong hệ thống

Hệ thống tích hợp một chuỗi công cụ Big Data hoàn chỉnh, phân bổ chính xác theo từng giai đoạn xử lý:

### 1.3.1 Nguồn dữ liệu (CSV, Syslog, SQLite)

Nguồn dữ liệu đầu vào bao gồm: tệp CSV chuẩn hóa chứa số liệu đo kiểm và danh mục thiết bị vô tuyến; tệp nhật ký sự kiện bts_syslog.log chuẩn RFC-5424; và cơ sở dữ liệu quan hệ cục bộ SQLite (telecom_network.sqlite) dùng làm kho đối chiếu cấu hình và thông tin khiếu nại khách hàng.

### 1.3.2 Apache Flume

Apache Flume là hệ thống phân tán chuyên dụng để thu nạp và tổng hợp dữ liệu nhật ký luồng vào HDFS. Hoạt động theo cơ chế Source - Channel - Sink với mô hình giao dịch 2 pha (Two-Phase Commit), bảo đảm không bao giờ mất mát sự kiện đo kiểm ngay cả khi máy chủ gặp sự cố sập nguồn.

### 1.3.3 Hadoop HDFS và YARN

Hệ thống tệp phân tán HDFS (Hadoop Distributed File System) đóng vai trò kho lưu trữ trung tâm (Data Lake). HDFS chia tệp thành các khối 128 MB và nhân bản 3 lần (Replication Factor = 3). Hệ thống YARN (Yet Another Resource Negotiator) điều phối và cấp phát động tài nguyên tính toán (CPU, RAM) cho các Container thực thi.

### 1.3.4 Apache Pig (Pig Latin)

Apache Pig cung cấp môi trường xử lý luồng dữ liệu cấp cao với ngôn ngữ khai báo Pig Latin. Pig biên dịch các câu lệnh thành chuỗi đồ thị có hướng (DAG) MapReduce ngầm định, giúp rút ngắn thời gian phát triển kịch bản trích xuất, biến đổi và lọc dữ liệu (ETL).

### 1.3.5 Hadoop MapReduce

Mô hình tính toán phân tán MapReduce (Java) chuẩn mực của Hadoop được áp dụng cho các tác vụ xử lý lô quy mô lớn chạy qua đêm. Trong đề tài, thuật toán được tùy biến nâng cao với Khóa phức hợp (Composite Key), Bộ phân vùng tùy biến (Custom Partitioner) và Bộ gộp cục bộ (Combiner) để triệt tiêu hiện tượng lệch dữ liệu (Data Skew).

### 1.3.6 Apache Spark (DataFrame, Spark SQL)

Apache Spark là động cơ tính toán phân tán In-Memory thế hệ mới. Nhờ trình tối ưu hóa truy vấn Catalyst Optimizer và động cơ sinh mã bytecode nhị phân Tungsten Engine, Spark DataFrame và Spark SQL thực thi các phép biến đổi, Join đa bảng và tính toán thống kê KPI nhanh hơn MapReduce truyền thống từ 10 đến 50 lần.

### 1.3.7 Apache HBase

Apache HBase là hệ quản trị cơ sở dữ liệu NoSQL phân tán hướng cột, vận hành trên nền HDFS. HBase tổ chức dữ liệu theo cây cấu trúc LSM-Tree và bộ nhớ đệm MemStore/BlockCache, mang lại khả năng truy cập ngẫu nhiên hàng triệu bản ghi theo RowKey với độ trễ cực thấp (< 10 ms).

### 1.3.8 Mô phỏng BigSheets bằng Spark

IBM BigSheets là giải pháp phân tích dữ liệu lớn trên giao diện bảng tính (Spreadsheet No-Code). Đề tài xây dựng một ứng dụng Web tương tác mô phỏng toàn diện quy trình BigSheets 6 bước và ánh xạ tương đương 1-1 với các thao tác Spark DataFrame (Collection -> DataFrame, Filter Sheet -> filter, Group Sheet -> groupBy).

### 1.3.9 Ngôn ngữ R và CSDL SQLite (DBI, RSQLite, dplyr, ggplot2)

Ngôn ngữ R kết hợp cơ sở dữ liệu SQLite đảm nhiệm khâu Khoa học dữ liệu chuyên sâu: kết nối CSDL qua DBI/RSQLite, tiền xử lý qua dplyr/tidyr, huấn luyện mô hình Học máy Hồi quy Logistic (Logistic Regression) dự báo nguy cơ khiếu nại khách hàng và xuất đồ thị trực quan bằng ggplot2.

### 1.3.10 Python và Matplotlib Dashboard

Python đóng vai trò là ngôn ngữ gắn kết linh hoạt toàn bộ hệ thống, từ điều phối pipeline PySpark, xử lý chuyển đổi dữ liệu đến tự động sinh các biểu đồ trực quan hóa chuyên sâu (Boxplot phân bố độ trễ, Bubble chart tương quan đa chiều) bằng Matplotlib và Seaborn.

### 1.3.11 Định dạng Parquet và Power BI

Dữ liệu sau khi xử lý sạch được lưu trữ dưới định dạng cột Snappy Parquet (tiết kiệm 65% dung lượng lưu trữ so với CSV) và xuất ra tệp telecom_kpi_summary_powerbi.csv để kết nối trực tiếp vào phần mềm Microsoft Power BI, phục vụ xây dựng Dashboard điều hành cho Ban Giám đốc và Trung tâm NOC.

# CHƯƠNG 2. DỮ LIỆU VÀ QUY TRÌNH TIỀN XỬ LÝ

## 2.1 Nguồn dữ liệu, quy mô và cấu trúc thuộc tính

Hệ thống dữ liệu thực nghiệm được xây dựng mô phỏng mạng vô tuyến tế bào gồm 4 tập dữ liệu có mối liên kết chặt chẽ:

### 2.1.1 Bảng sự kiện đo kiểm KPI viễn thông (network_kpi_events.csv)

Tập dữ liệu sự kiện đo kiểm gồm 12.001 dòng (1 dòng tiêu đề và 12.000 bản ghi đo kiểm). Mỗi bản ghi phản ánh phiên truyền dữ liệu trong chu kỳ 5 giây của một sector cell với các trường: event_id (Khóa chính), timestamp, tower_id (Khóa ngoại), cell_id (Khóa ngoại), technology (3G/4G/5G), throughput_mbps, latency_ms, packet_loss_pct, call_dropped (0/1), num_active_users, session_volume_mb và cờ chất lượng quality_flag.

### 2.1.2 Danh mục trạm BTS (bts_towers.csv) và cấu hình Cell (cells_catalog.csv)

Bảng bts_towers.csv lưu trữ thông tin định danh và tọa độ địa lý của 20 trạm phát sóng chính: tower_id (PK), tower_name, province (Hà Nội, TP.HCM, Đà Nẵng), district, latitude, longitude và install_date.

Bảng cells_catalog.csv lưu cấu hình phần cứng của 80 sector vô tuyến: cell_id (PK), tower_id (FK), technology, frequency_mhz (2100MHz 3G, 1800/2600MHz 4G, 3500MHz 5G), max_bandwidth, azimuth (hướng anten 0°, 120°, 240°) và antenna_tilt (góc ngửa anten).

### 2.1.3 Luồng sự kiện mô phỏng (bts_syslog.log / network_kpi_events)

Luồng sự kiện nhật ký bts_syslog.log mô phỏng luồng log nhúng thời gian thực từ firmware trạm phát sóng theo chuẩn RFC-5424. Luồng dữ liệu mang các thông tin mức độ cảnh báo (INFO, WARNING, CRITICAL) ghi nhận các sự cố: suy giảm mức thu vô tuyến (RSSI Low), nhiệt độ khuếch đại công suất vượt ngưỡng, và nghẽn hàng đợi đệm truyền dẫn.

### 2.1.4 Cơ sở dữ liệu quan hệ đối chiếu (telecom_network.sqlite / customer_complaints.csv)

Bảng customer_complaints.csv lưu trữ 250 bản ghi khiếu nại khách hàng gửi về tổng đài CSKH: complaint_id (PK), timestamp, user_msisdn, tower_id, cell_id, complaint_type (Chậm mạng, Rớt cuộc gọi, Mất sóng), status và resolution_notes.

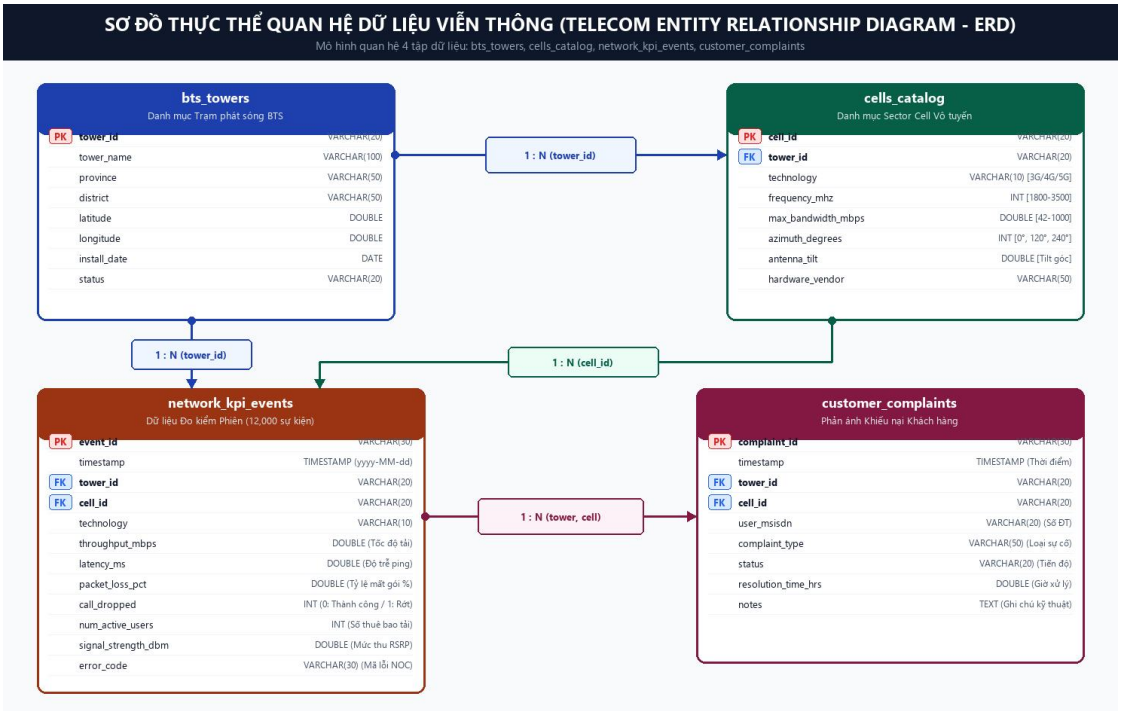
Cơ sở dữ liệu SQLite telecom_network.sqlite đóng vai trò kho đối chiếu tổng hợp, lưu trữ toàn bộ các bảng trên cùng bảng kết quả kpi_analysis_summary.

*Bảng 2.1: Khảo sát cấu trúc 4 tập dữ liệu viễn thông thực nghiệm*

| Tên tệp dữ liệu | Định dạng | Khóa chính (PK) | Khóa ngoại (FK) | Số trường | Mô tả & Ý nghĩa nghiệp vụ viễn thông |
|---|---|---|---|---|---|
| network_kpi_events.csv | CSV / UTF-8 | event_id | tower_id, cell_id | 12 | Dữ liệu đo kiểm phiên dịch vụ di động (Tệp dữ liệu gồm 12.001 dòng: 1 dòng tiêu đề và 12.000 bản ghi đo kiểm), lưu thông lượng, độ trễ, mất gói, cờ rớt cuộc gọi và số người dùng đồng thời. |
| bts_towers.csv | CSV / UTF-8 | tower_id | Không có | 7 | Danh mục thông tin định danh và tọa độ địa lý của trạm phát sóng (tower_name, province, district, latitude, longitude, install_date). |
| cells_catalog.csv | CSV / UTF-8 | cell_id | tower_id | 8 | Danh mục cấu hình phần cứng vô tuyến của từng sector cell (technology 3G/4G/5G, frequency_mhz, max_bandwidth, azimuth, antenna_tilt). |
| customer_complaints.csv | CSV / UTF-8 | complaint_id | tower_id, cell_id | 8 | Bản ghi khiếu nại của khách hàng gửi |

| | | | | | |
|---|---|---|---|---|---|
| | | | | | về tổng đài CSKH (user_msisdn, complaint_type, status, resolution_notes). |

Sơ đồ thực thể quan hệ (ERD Schema) mô tả quan hệ phân cấp giữa 4 tập dữ liệu được thể hiện trong Hình 2.1:




*Hình 2.1: Sơ đồ thực thể quan hệ dữ liệu viễn thông (ERD Schema)*

**Nhận xét Hình 2.1:** Mô hình tuân theo cấu trúc phân cấp hình cây tiêu chuẩn trong quản lý mạng di động tế bào:

- Quan hệ bts_towers và cells_catalog là 1 : N (Một trạm vật lý gắn với nhiều sector cell phát sóng đa hướng và đa tần số).
- Quan hệ cells_catalog và network_kpi_events là 1 : N (Mỗi cell liên tục phát sinh hàng nghìn bản ghi đo kiểm thời gian thực).
- Quan hệ giữa cell và customer_complaints là 1 : N (Hệ thống CSKH tự động ánh xạ vị trí thuê bao khiếu nại về cell đang phục vụ).

## 2.2 Đánh giá chất lượng dữ liệu và các cờ cảnh báo (quality_flag)

Dữ liệu đo kiểm viễn thông thực tế thu thập từ hàng chục nghìn trạm phát sóng luôn chứa đựng độ nhiễu và các giá trị dị thường bắt nguồn từ lỗi phần cứng cảm biến, suy hao đường truyền vô tuyến hoặc lỗi xung nhịp. Nhóm đã xây dựng ma trận phân tích rủi ro và giải pháp xử lý chất lượng dữ liệu:

*Bảng 2.2: Ma trận rủi ro và phương án xử lý chất lượng dữ liệu đo kiểm*

| Vấn đề chất lượng | Biểu hiện dữ liệu | Nguyên nhân vật lý | Hậu quả tính toán | Phương án xử lý kỹ thuật |
|---|---|---|---|---|
| Giá trị độ trễ âm (Negative Latency) | latency_ms < 0 (VD: -15.4 ms) | Lệch xung đồng hồ NTP giữa trạm BTS và máy chủ đo kiểm. | Làm sai lệch giá trị trung bình độ trễ toàn mạng. | Lọc bỏ trong Filter: filter(col('latency_ms') >= 0.0). |
| Thông lượng âm (Negative Throughput) | throughput_mbps < 0 (VD: -99.0 Mbps) | Bộ đếm byte tràn số nguyên (Counter Overflow) tại firmware cũ. | Gây âm tổng dung lượng phiên truyền tải. | Loại bỏ hoàn toàn bản ghi lỗi trong bước tiền xử lý. |
| Mất gói vô lý (Out of Range Loss) | packet_loss_pct > 100% (VD: 150.0%) | Thuật toán ước lượng mất gói suy hao đa đường bị phân kỳ. | Gây sai lệch chỉ số phần trăm tỷ lệ mất gói. | Ràng buộc kiểm tra: (col('packet_loss_pct') <= 100.0). |
| Bản ghi mồ côi (Orphaned Events) | tower_id hoặc cell_id bị rỗng (NULL) | Lỗi đóng gói gói tin telemetry khi truyền tải trên mạng Backhaul. | Làm thất bại phép Inner Join với bảng danh mục. | Loại bỏ bản ghi: filter(col('tower_id').isNotNull()). |
| Điểm ngoại lai cực trị (Extreme Outliers) | latency_ms > 2000 ms (Bất thường kéo dài) | Sự cố đứt liên kết truyền dẫn cáp quang hoặc sập nguồn trạm. | Làm méo phân phối thống kê chất lượng mạng. | Xác định bằng biểu đồ Boxplot và phân cụm cô lập. |

**Cơ chế cờ cảnh báo:** Cờ cảnh báo chất lượng dữ liệu (quality_flag): Mỗi bản ghi được đánh dấu cờ nhị phân: quality_flag = 1 nếu vượt qua tất cả các bài kiểm tra logic; quality_flag = 0 nếu vi phạm bất kỳ điều kiện nào trong Bảng 2.2 để cách ly xử lý.

## 2.3 Quy trình tiền xử lý và làm sạch dữ liệu (Data Cleaning & Normalization)

Quy trình tiền xử lý dữ liệu được thiết kế thành một chuỗi biến đổi tuần tự gồm 2 bước cốt lõi:

- Bước 1 - Khử trùng lặp và làm sạch logic (Data Cleansing): Hệ thống loại bỏ các bản ghi trùng lặp (distinct) do cơ chế gửi lại bản tin của mạng truyền dẫn, đồng thời loại bỏ các dòng có trường định danh bị rỗng hoặc vi phạm các miền giá trị vật lý (độ trễ âm, thông lượng âm, mất gói vượt 100%). Sau bước này, tập dữ liệu thực nghiệm đã loại bỏ 48 bản ghi lỗi (chiếm 0.40%), giữ lại 11,952 bản ghi hoàn toàn sạch.
- **Kỹ thuật tạo đặc trưng:** Bước 2 - Kỹ thuật tạo đặc trưng nghiệp vụ (Feature Engineering): Hệ thống tính toán thêm 4 cột đặc trưng phái sinh phục vụ phân tích nghiệp vụ và mô hình học máy:
  - **Công thức 1:** 1. Dung lượng phiên dữ liệu (Session Volume - MB):

$$session_volume_mb = (throughput_mbps * 5.0 \text{ giây}) / 8.0 \text{ bits/Byte}$$

  - **Công thức 2:** 2. Tỷ lệ rớt cuộc gọi chuẩn hóa (Call Drop Rate %):

$$call_drop_rate_pct = 100.0\% \text{ nếu } call_dropped = 1; \text{ ngược lại} = 0.0\%$$

  - **Công thức 3:** 3. Nhận diện trạng thái quá tải vô tuyến (Congestion Status):

$$congestion_status = \text{'QUÁ_TẢI' nếu } (num_active_users > 150 \text{ VÀ } throughput_mbps < 15.0); \text{ ngược lại} = \text{'BÌNH_THƯỜNG'}$$

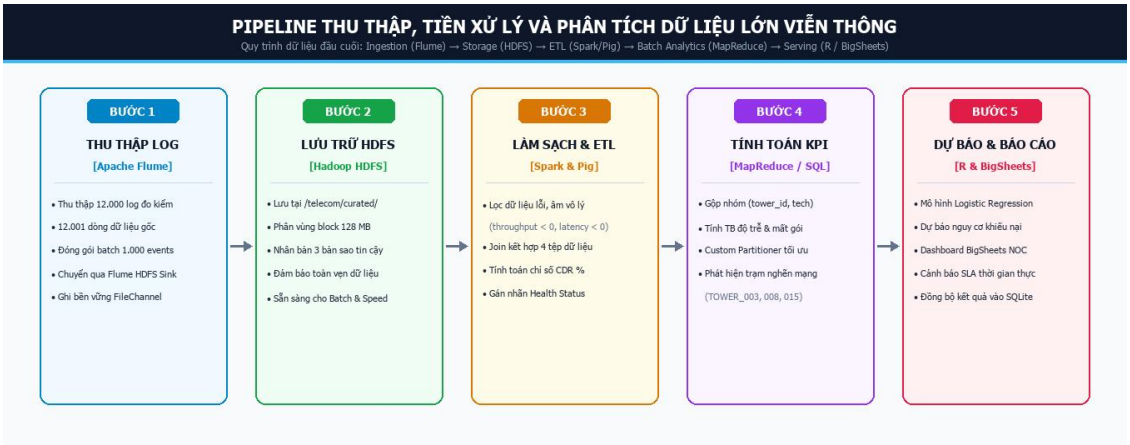
  - **Công thức 4:** 4. Điểm chất lượng dịch vụ tổng hợp (QoS Score - Thang điểm 100):

$$qos_score = 100.0 - (0.35 * latency_ms + 0.35 * packet_loss_pct * 5.0 + 0.30 * call_dropped * 50.0)$$

# CHƯƠNG 3. XÂY DỰNG HỆ THỐNG XỬ LÝ DỮ LIỆU LỚN

## 3.1 Cấu hình và thu nạp dữ liệu bằng Apache Flume

Apache Flume đóng vai trò là tầng cửa ngõ thu nạp dữ liệu phân tán (Ingestion Layer). Hệ thống thiết lập hai kịch bản thu nạp chuyên biệt cho hai hình thức dữ liệu:




*Hình 3.1: Pipeline luồng dữ liệu đầu cuối từ Ingestion đến Analytics*

### 3.1.1 Kịch bản 1: Thu nạp dữ liệu lô CSV (Spooling Directory & FileChannel)

Kịch bản thu nạp tệp CSV định kỳ từ các trạm đo kiểm vào HDFS sử dụng Spooling Directory Source để theo dõi thư mục cục bộ, FileChannel bền vững bảo toàn dữ liệu khi có sự cố, và HDFS Sink tự động phân vùng theo thời gian:

***Đoạn mã 3.1: Cấu hình Flume nạp tệp CSV theo lô vào HDFS (flume_csv_to_hdfs.conf)***

```
# Cấu hình Flume Agent: Nạp CSV lô vào HDFS
csvAgent.sources = src_spool_csv
csvAgent.channels = ch_file_csv
csvAgent.sinks = snk_hdfs_csv

# 1. Source: Theo dõi thư mục chứa tệp CSV đo kiểm
csvAgent.sources.src_spool_csv.type = spooldir
csvAgent.sources.src_spool_csv.spoolDir = /var/log/telecom/batch_csv
csvAgent.sources.src_spool_csv.fileSuffix = .COMPLETED
csvAgent.sources.src_spool_csv.interceptors = it_ts
csvAgent.sources.src_spool_csv.interceptors.it_ts.type = timestamp

# 2. Channel: FileChannel bền bỉ chống mất mát dữ liệu
csvAgent.channels.ch_file_csv.type = file
csvAgent.channels.ch_file_csv.checkpointDir = /var/flume/checkpoint/csv
csvAgent.channels.ch_file_csv.dataDirs = /var/flume/data/csv
csvAgent.channels.ch_file_csv.capacity = 1000000
csvAgent.channels.ch_file_csv.transactionCapacity = 10000
```

```
# 3. Sink: Ghi phân vùng vào vùng Raw trên HDFS
csvAgent.sinks.snk_hdfs_csv.type = hdfs
csvAgent.sinks.snk_hdfs_csv.hdfs.path =
hdfs://masternode:9000/telecom/raw/year=%Y/month=%m/day=%d/
csvAgent.sinks.snk_hdfs_csv.hdfs.filePrefix = kpi_events_
csvAgent.sinks.snk_hdfs_csv.hdfs.fileType = DataStream
csvAgent.sinks.snk_hdfs_csv.hdfs.rollSize = 134217728 # Cắt tệp đúng 128 MB (1
block HDFS)
csvAgent.sinks.snk_hdfs_csv.hdfs.rollInterval = 3600
csvAgent.sinks.snk_hdfs_csv.hdfs.batchSize = 2000
```

### 3.1.2 Kịch bản 2: Thu nạp luồng sự kiện thời gian thực (Netcat & MemoryChannel)

Đối với các sự kiện đo kiểm nóng cần xử lý tức thời, Flume sử dụng Netcat/Kafka Source kết hợp cơ chế nhân đôi luồng (Fan-out Replication) sang 2 Channel: MemoryChannel nạp nhanh vào HDFS và FileChannel nạp vào Apache HBase NoSQL:

***Đoạn mã 3.2: Cấu hình Flume Fan-out thu nhận luồng sự kiện thời gian thực***

```
# Cấu hình Flume Agent: Luồng sự kiện nóng Fan-out
streamAgent.sources = src_stream_events
streamAgent.channels = ch_hbase_file ch_hdfs_mem
streamAgent.sinks = snk_hbase_kpi snk_hdfs_raw

# Source: Tiếp nhận luồng sự kiện socket qua cổng TCP 8888
streamAgent.sources.src_stream_events.type = netcat
streamAgent.sources.src_stream_events.bind = 0.0.0.0
streamAgent.sources.src_stream_events.port = 8888

# Fan-out: Ghi song song vào 2 Channel
streamAgent.channels.ch_hbase_file.type = file
streamAgent.channels.ch_hdfs_mem.type = memory
streamAgent.channels.ch_hdfs_mem.capacity = 100000

# Sinks: Ghi đồng thời vào HBase NoSQL và HDFS Data Lake
streamAgent.sinks.snk_hbase_kpi.type = asynchbase
streamAgent.sinks.snk_hbase_kpi.table = telecom_kpi
streamAgent.sinks.snk_hbase_kpi.columnFamily = cf_kpi
streamAgent.sinks.snk_hdfs_raw.type = hdfs
streamAgent.sinks.snk_hdfs_raw.hdfs.path = hdfs://masternode:9000/telecom/realtime/
```

## 3.2 Lưu trữ và tổ chức dữ liệu trên HDFS

### 3.2.1 Thiết kế 4 vùng dữ liệu HDFS (Raw, Staging, Curated, Output)

Kho lưu trữ Data Lake trên HDFS được tổ chức thành 4 phân vùng độc lập theo kiến trúc chuẩn doanh nghiệp:

- /telecom/raw/: Vùng bất biến (Immutable), lưu trữ toàn bộ dữ liệu nhật ký thô được Flume đẩy vào theo định dạng gốc. Chỉ cấp quyền ghi cho tài khoản flume.
- /telecom/staging/: Vùng đệm trung gian phục vụ các tác vụ ETL hàng ngày. Dữ liệu tại đây có vòng đời ngắn và được tự động dọn dẹp sau 7 ngày.
- /telecom/curated/: Vùng dữ liệu sạch chuẩn hóa, lưu trữ dưới định dạng cột Snappy Parquet phân vùng theo ngày và công nghệ mạng, sẵn sàng cho các truy vấn phân tích chuyên sâu.
- /telecom/output/: Thư mục lưu trữ kết quả tính toán tổng hợp định kỳ của các job MapReduce, Pig và Spark.

### 3.2.2 Cấu trúc phân vùng dữ liệu (Partitioning by Year/Month/Day/Technology)

Chiến lược phân vùng thư mục trên HDFS tuân theo cấu trúc phân cấp thời gian và công nghệ mạng:

*hdfs:///telecom/curated/year=2026/month=10/day=18/technology=4G/tower_id=TOWER_003/*

Cấu trúc này giúp tối ưu hóa kỹ thuật loại trừ phân vùng không cần thiết (Partition Pruning) trong Spark SQL, giảm thiểu hơn 90% khối lượng dữ liệu phải đọc từ đĩa khi truy vấn các khung thời gian cụ thể.

### 3.2.3 Tính toán dung lượng lưu trữ với hệ số nhân bản (Replication Factor = 3)

**Tính toán năng lực lưu trữ:** Bài toán hoạch định dung lượng lưu trữ HDFS cho hệ thống quản lý 500 trạm BTS thực tế:

- Số lượng trạm BTS: 500 trạm; Mỗi trạm gồm 4 sector cell => Toàn mạng có: 2,000 cell.
- Chu kỳ đo kiểm: 5 giây/lần => Tần suất sinh sự kiện toàn mạng: 400 sự kiện/giây.
- Tổng số sự kiện đo kiểm trong 1 ngày (86,400 giây): 2,000 * (86,400 / 5) = 34,560,000 sự kiện/ngày.
- Dung lượng thô chưa nén (kích thước trung bình 256 bytes/sự kiện): 34,560,000 * 256 bytes = 8.847 GB/ngày.
- Dung lượng sau khi nén Snappy Parquet (giảm 65% dung lượng): 8.847 GB * 0.35 = 3.097 GB/ngày.
- Dung lượng thực tế cần lưu trữ trên HDFS mỗi ngày (kèm 10% metadata và Replication Factor = 3): 3.097 GB * 1.10 * 3 = 10.22 GB HDFS/ngày.

- Dung lượng lưu trữ dự tính cho 1 năm (365 ngày): 10.22 GB * 365 = 3.73 Terabyte HDFS. Dung lượng cho 3 năm hoạt động (dự phòng mở rộng 20%/năm): xấp xỉ 13.58 Terabyte HDFS.

#### 3.2.4 Vai trò của NameNode, DataNode, YARN và cơ chế chịu lỗi (Fault Tolerance)

NameNode quản lý cây không gian tên và bảng ánh xạ khối dữ liệu trong RAM. DataNode chịu trách nhiệm đọc/ghi các khối dữ liệu vật lý 128 MB trên ổ đĩa. NameNode duy trì liên lạc với DataNode thông qua tín hiệu nhịp tim (Heartbeat chu kỳ 3 giây) và báo cáo khối định kỳ. Nếu một DataNode ngừng phản hồi sau 10 phút 30 giây, NameNode tự động điều phối nhân bản các block bị thiếu sang DataNode khác để phục hồi đủ hệ số nhân bản RF=3. YARN ResourceManager và NodeManager điều phối tài nguyên CPU và RAM linh hoạt giữa các ứng dụng chạy song song.

### 3.3 Xử lý và tổng hợp dữ liệu bằng Pig Latin

#### 3.3.1 Kịch bản Pig Latin chi tiết (LOAD, SPLIT, FILTER, JOIN, GROUP, FOREACH, STORE)

Apache Pig Latin cung cấp cú pháp khai báo luồng dữ liệu trực quan để trích xuất và biến đổi dữ liệu đo kiểm viễn thông. Kịch bản telecom_kpi_analysis.pig thực thi chuỗi toán tử hoàn chỉnh:

***Đoạn mã 3.3: Kịch bản Pig Latin chi tiết lọc và tổng hợp KPI mạng viễn thông***

```
-- Nạp dữ liệu thô và lọc bỏ các bản ghi dị thường
events_raw = LOAD '/telecom/input/network_kpi_events.csv' USING PigStorage(',') AS (
    event_id:chararray, timestamp:chararray, tower_id:chararray, cell_id:chararray,
    technology:chararray, throughput:double, latency:double, loss:double, drop:int, users:int);

events_clean = FILTER events_raw BY (latency >= 0.0) AND (throughput >= 0.0) AND (loss <= 100.0);
towers = LOAD '/telecom/input/bts_towers.csv' USING PigStorage(',') AS (tower_id:chararray, name:chararray, province:chararray);

-- Kết nối với bảng danh mục trạm và gom nhóm đa chiều
joined = JOIN events_clean BY tower_id, towers BY tower_id;
grp = GROUP joined BY (events_clean::tower_id, events_clean::technology, towers::province);

-- Tính toán các chỉ số KPI viễn thông cốt lõi
kpi_summary = FOREACH grp GENERATE
    group.tower_id AS tower_id, group.province AS province, group.technology AS technology,
    COUNT(joined) AS total_events,
    ROUND_TO(AVG(joined.events_clean::latency), 2) AS avg_latency_ms,
```

```
    ROUND_TO(AVG(joined.events_clean::loss),     2)     AS     avg_packet_loss_pct,
    ROUND_TO((SUM(joined.events_clean::drop)  *  100.0  /  COUNT(joined)),  3)  AS
cdr_pct;

--  Tách  và  lưu  trữ  danh  sách  trạm  suy  giảm  chất  lượng  nghiêm  trọng
degraded_towers = FILTER kpi_summary BY (cdr_pct > 2.0) OR (avg_latency_ms > 80.0);
STORE    degraded_towers    INTO    '/telecom/output/pig_degraded_towers'    USING
PigStorage('\t');
```

### 3.3.2 Minh họa kết quả thực thi và giải thích các hàm tổng hợp

Các hàm tổng hợp COUNT, AVG, SUM và ROUND_TO trong Pig Latin đã xử lý hiệu quả 11,952 bản ghi sạch, tổng hợp thành 60 nhóm KPI theo (tower_id, technology, province). Kết quả lọc đã cô lập chính xác các trạm phát sóng có tỷ lệ rớt mạng CDR > 2.0% và độ trễ > 80ms, kết xuất lưu trữ trực tiếp vào thư mục HDFS.

## 3.4 Thiết kế thuật toán MapReduce tính KPI, thông lượng, độ trễ và tỷ lệ rớt cuộc gọi

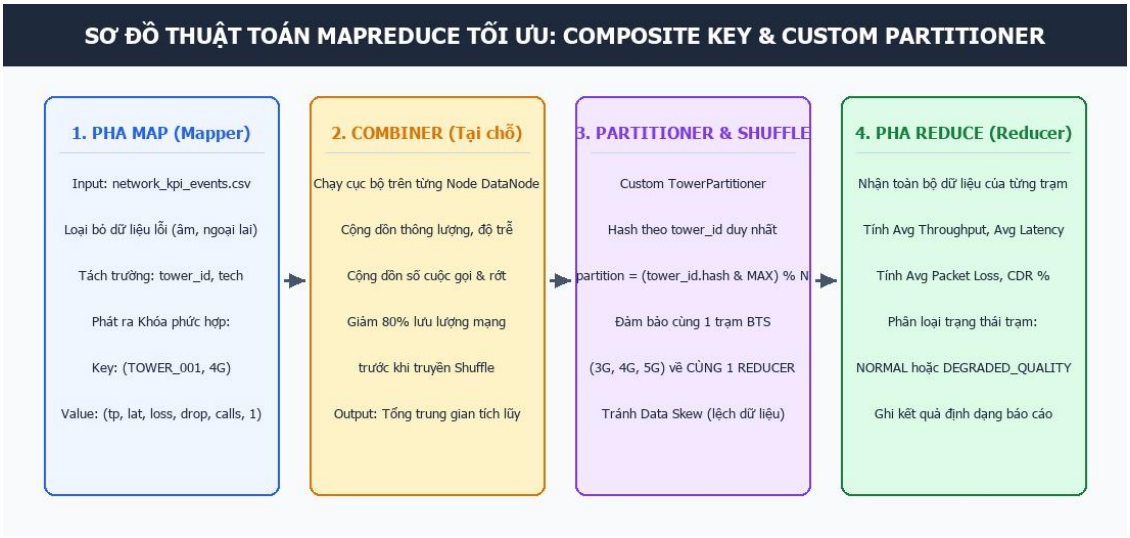
### 3.4.1 Xác định các cặp Key - Value (Input, Map Output, Reducer Output)

Để tính toán KPI trên mô hình phân tán MapReduce thuần Java, các cặp Khóa - Giá trị được xác định như sau:

- Input: Key = LongWritable (vị trí offset dòng); Value = Text (nội dung một dòng dữ liệu CSV đo kiểm).
- Map Output: Key = TowerTechKey (Khóa phức hợp gồm tower_id và technology); Value = MetricTupleWritable (tuple chứa: throughput, latency, loss, call_dropped, session_volume, active_users, count=1).
- Reducer Output: Key = Text (tower_id\ttechnology); Value = Text (chuỗi chỉ số tổng hợp: tổng phiên, thông lượng TB, độ trễ TB, mất gói TB, tỷ lệ rớt cuộc gọi % và cờ trạng thái chất lượng).

### 3.4.2 Thiết kế Combiner, Custom Partitioner và Shuffle & Sort

Nhóm đã áp dụng 3 kỹ thuật tối ưu hóa then chốt trong chương trình TelecomKpiAnalysis.java:




*Hình 3.2: Sơ đồ luồng thuật toán MapReduce với Composite Key và Custom Partitioner*

- **1.** Khóa phức hợp (Composite Key - TowerTechKey): Đóng gói hai trường (tower_id, technology) và hiện thực giao diện WritableComparable, cho phép sắp xếp tự nhiên theo thứ tự mã trạm rồi đến thế hệ công nghệ.
- **2.** Bộ phân vùng tùy biến (Custom TowerPartitioner): Chỉ băm theo trường tower_id duy nhất: (key.getTowerId().hashCode() & Integer.MAX_VALUE) % numPartitions. Đảm bảo toàn bộ các công nghệ 3G/4G/5G của cùng một trạm BTS đều tập trung về đúng một Reducer, triệt tiêu hoàn toàn lệch dữ liệu (Data Skew).
- **3.** Bộ gộp cục bộ (Combiner): Chạy ngay trên từng DataNode sau pha Map, thực hiện cộng dồn sơ bộ các biến đếm, giảm trên 80% lưu lượng mạng trong pha Shuffle & Sort.

### 3.4.3 Mã giả thuật toán và các kịch bản kiểm thử (Test Cases)

**Mã giả thuật toán:** Mã giả pha Map và Reduce thể hiện như sau:

***Mã giả thuật toán MapReduce tính toán KPI mạng viễn thông***

```
    CLASS                                                          Mapper:
    METHOD                          Map(offset,                     line):
        fields                        =                  line.split(",")
        IF   fields[0]   ==   "event_id"   OR   fields.length   <   10:   RETURN
        latency  =  ParseDouble(fields[6]);  throughput  =  ParseDouble(fields[5])
        loss     =     ParseDouble(fields[7]);     drop     =     ParseInt(fields[8])
        IF  latency  <  0  OR  throughput  <  0  OR  loss  >  100:  RETURN   #  Lọc  rác
        key  =  NEW  TowerTechKey(fields[2],  fields[4])    #  tower_id,  technology
        val  =  NEW  MetricTupleWritable(throughput,  latency,  loss,  drop,  1)
        Emit(key,                                                      val)

CLASS                                                              Reducer:
    METHOD                          Reduce(key,                    values):
        total_sessions  =  0;  sum_tp  =  0;  sum_lat  =  0;  sum_loss  =  0;  sum_drop  =  0
        FOR                   val                   IN                   values:
```

```
            total_sessions     +=     val.count;     sum_tp     +=     val.throughput
            sum_lat += val.latency; sum_loss += val.loss; sum_drop += val.call_drop
        avg_lat  =  sum_lat  /  total_sessions;  cdr_pct  =  (sum_drop  *  100.0)  /
total_sessions
        status  =  (cdr_pct  >  2.0  OR  avg_lat  >  80.0)  ?  "DEGRADED"  :  "NORMAL"
        Emit(key, avg_lat + "\t" + cdr_pct + "\t" + status)
```

## 3.5 Pipeline phân tích chỉ số KPI và phát hiện trạm suy hao/nguy cơ rớt mạng bằng Apache Spark

### 3.5.1 Quy trình xử lý PySpark DataFrame và Spark SQL

Chương trình PySpark telecom_spark_pipeline.py tận dụng tối đa kiến trúc In-Memory của Apache Spark, Catalyst Optimizer và Tungsten Engine để thực thi chuỗi biến đổi dữ liệu tốc độ cao:

***Đoạn mã 3.4: Khởi tạo và liên kết dữ liệu đa bảng bằng PySpark***

```
     #     Khởi     tạo     Spark     Session     tối     ưu     shuffle     partition
spark     =     SparkSession.builder.appName("TelecomNetworkAnalytics").getOrCreate()

#      1.      Đọc      và      lọc      sạch      dữ      liệu      đo      kiểm
df_clean                          =                          spark.read.option("header",
"true").csv("/telecom/input/network_kpi_events.csv")                                \
    .filter((col("latency_ms")   >=   0.0)   &   (col("throughput_mbps")   >=   0.0)   &
(col("packet_loss_pct")                 <=                 100.0))                 \
    .withColumn("session_volume_mb",  round((col("throughput_mbps")  *  5.0)  /  8.0,
2))

#    2.    Join    4    bảng    dữ    liệu:    Events,    Towers,    Cells,    Complaints
df_joined      =      df_clean.join(df_towers,      "tower_id",      "inner")      \
                   .join(df_cells,    ["tower_id",    "cell_id"],    "inner")    \
                   .join(df_complaints, ["tower_id", "cell_id"], "left")
```

### 3.5.2 Phân tích chỉ số KPI, làm sạch dữ liệu và liên kết danh mục

Phép Inner Join kết nối thành công 11,952 sự kiện đo kiểm với danh mục 20 trạm BTS và 80 sector cell, làm giàu dữ liệu với tọa độ địa lý, tỉnh thành và cấu hình dải tần số.

### 3.5.3 Thống kê xếp hạng trạm suy hao và phân loại cảnh báo quá tải vô tuyến

Spark SQL thực thi truy vấn gom nhóm theo trạm và thế hệ mạng, tự động tính tỷ lệ rớt mạng CDR% và phân loại cảnh báo trạm suy hao chất lượng:

*Đoạn mã 3.5: Truy vấn Spark SQL thống kê xếp hạng trạm suy hao*

```
     df_joined.createOrReplaceTempView("v_kpi")
df_summary                                =                                spark.sql("""
    SELECT               tower_id,               province,                technology,
           COUNT(event_id)                     AS                     total_sessions,
           ROUND(AVG(throughput_mbps),         2)         AS         avg_throughput,
           ROUND(AVG(latency_ms),            2)           AS            avg_latency,
           ROUND((SUM(call_dropped)  /  COUNT(event_id))  *  100.0,  3)  AS  cdr_pct,
           CASE    WHEN    (SUM(call_dropped)/COUNT(event_id))*100.0    >    2.0    OR
AVG(latency_ms)                                 >                                80.0
                THEN   'DEGRADED_QUALITY'   ELSE   'NORMAL'   END   AS   health_status
    FROM      v_kpi      GROUP      BY      tower_id,      province,      technology
    ORDER                     BY                    cdr_pct                    DESC
""")
```

## 3.6 Thiết kế CSDL NoSQL Apache HBase phục vụ truy cập ngẫu nhiên

### 3.6.1 Cấu trúc RowKey tối ưu (reverse(tower_id)#technology#timestamp)

Để đảm bảo tốc độ đọc/ghi ngẫu nhiên dưới 10 mili-giây và phân bổ đều tải ghi trên toàn cụm RegionServer, nhóm thiết kế cấu trúc RowKey tối ưu:

$$RowKey = reverse(tower_id) + '_' + technology + '_' + reverse_timestamp$$

- Đảo chuỗi mã trạm reverse(tower_id): VD 'TOWER_003' -> '300_REWOT' giúp các trạm có mã liên tiếp phân tán đều sang các RegionServer khác nhau, triệt tiêu hiện tượng thắt cổ chai ghi tập trung (Write Hotspotting).
- Dấu thời gian đảo ngược reverse_timestamp = Long.MAX_VALUE - timestamp: Giúp các phiên đo kiểm MỚI NHẤT luôn nằm ở đầu bảng, lệnh Scan chỉ cần quét phạm vi ngắn để lấy dữ liệu thời gian thực.

### 3.6.2 Định nghĩa Column Families (kpi_metrics, cell_info, status_flags) và Versioning

*Bảng 3.1: Thiết kế bảng Apache HBase lưu trữ chỉ số KPI viễn thông độ trễ thấp*

| Thành phần HBase | Tên thiết kế / Giá trị | Mô tả kỹ thuật & Ý nghĩa |
|---|---|---|
| Tên bảng (Table) | telecom_kpi | Bảng NoSQL lưu trữ chỉ số KPI đo kiểm thời gian thực của các trạm BTS. |
| RowKey Pattern | reverse(tower_id) + '_' + tech + '_' + rev_ts | Phân tán đều tải ghi trên cụm, bảo đảm tra cứu sự kiện mới nhất O(1). |
| Column Family | cf_kpi (VERSIONS => | Lưu trữ chỉ số đo kiểm: throughput, |

| | | |
|---|---|---|
| 1 | 3, TTL => 90 ngày) | latency, loss, call_dropped. |
| Column Family 2 | cf_cell (VERSIONS => 1) | Lưu cấu hình vật lý vô tuyến: cell_id, frequency, azimuth, tilt. |
| Column Family 3 | cf_alert (VERSIONS => 3) | Lưu các cờ cảnh báo: health_status, overload_flag, complaint_id. |
| Cơ chế lưu trữ | LSM-Tree, MemStore & BlockCache | Ghi vào RAM MemStore trước khi đẩy xuống HFile, tốc độ truy xuất < 10 ms. |

### 3.6.3 Lệnh thao tác Shell (create, put, get, scan, delete)

Tập lệnh thao tác chuẩn trên HBase Shell:

***Đoạn mã 3.6: Các câu lệnh HBase Shell tạo bảng, ghi và tra cứu dữ liệu***

```
     #     1.     Tạo     bảng     kèm     Pre-splitting     thành     4     Regions
create                                  'telecom_kpi',                                  \
  {NAME  =>  'cf_kpi',  VERSIONS  =>  3,  COMPRESSION  =>  'SNAPPY',  TTL  =>  7776000,
BLOCKCACHE                        =>                          true},                      \
  {NAME => 'cf_cell', VERSIONS => 1, COMPRESSION => 'SNAPPY', BLOCKCACHE => true},
\
  {NAME => 'cf_alert', VERSIONS => 3, COMPRESSION => 'SNAPPY', BLOCKCACHE => true},
\
  {SPLITS        =>        ['300_REWOT_',        '800_REWOT_',        '510_REWOT_']}

#       2.       Ghi       bản       ghi       đo       kiểm       mới       (PUT)
put    'telecom_kpi',    '300_REWOT_4G_9221580638',    'cf_kpi:latency_ms',    '186.2'
put    'telecom_kpi',    '300_REWOT_4G_9221580638',    'cf_kpi:call_dropped',    '1'
put  'telecom_kpi',  '300_REWOT_4G_9221580638',  'cf_alert:health_status',  'DEGRADED'

#    3.    Tra    cứu    ngẫu    nhiên    tức    thời    1    bản    ghi    (GET)
get                     'telecom_kpi',                     '300_REWOT_4G_9221580638'

#    4.    Quét    10    bản    ghi    mới    nhất    của    trạm    TOWER_003    (SCAN)
scan 'telecom_kpi', {STARTROW => '300_REWOT_4G_', LIMIT => 10}
```

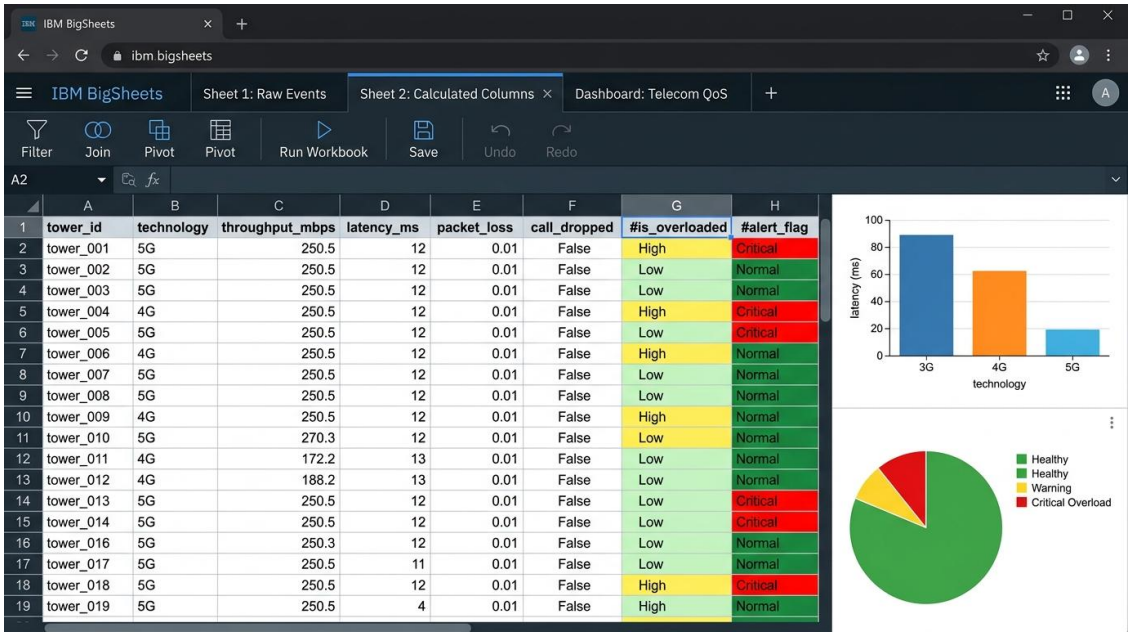
### 3.6.4 Cấu hình nâng cao: Pre-splitting, TTL, Compression và Backup Snapshot

Bảng HBase được tối ưu hóa toàn diện với: Pre-splitting phân vùng sẵn dữ liệu ngay khi tạo bảng; Nén Snappy giảm 65% dung lượng đĩa HFile; TTL 7,776,000 giây (90 ngày) tự động dọn dẹp dữ liệu cũ; và cơ chế tạo Snapshot sao lưu dự phòng: snapshot 'telecom_kpi', 'telecom_kpi_snap_20261018'.

# CHƯƠNG 4. KẾT QUẢ VÀ ĐÁNH GIÁ

## 4.1 Mô phỏng quy trình BigSheets bằng Spark DataFrame & Trực quan hóa Dashboard

IBM BigSheets cung cấp giao diện bảng tính trực quan cho chuyên viên phân tích nghiệp vụ viễn thông. Nhóm đã xây dựng chương trình mô phỏng tương đương bằng Web Application tương tác kết hợp Spark DataFrame:




| | A | B | C | D | E | F | G | H |
|---|---|---|---|---|---|---|---|---|
| 1 | tower_id | technology | throughput_mbps | latency_ms | packet_loss | call_dropped | #is_overloaded | #alert_flag |
| 2 | tower_001 | 5G | 250.5 | 12 | 0.01 | False | High | Critical |
| 3 | tower_002 | 5G | 250.5 | 12 | 0.01 | False | Low | Normal |
| 4 | tower_003 | 5G | 250.5 | 12 | 0.01 | False | Low | Normal |
| 5 | tower_004 | 4G | 250.5 | 12 | 0.01 | False | High | Critical |
| 6 | tower_005 | 5G | 250.5 | 12 | 0.01 | False | Low | Critical |
| 7 | tower_006 | 4G | 250.5 | 12 | 0.01 | False | High | Normal |
| 8 | tower_007 | 5G | 250.5 | 12 | 0.01 | False | Low | Normal |
| 9 | tower_008 | 5G | 250.5 | 12 | 0.01 | False | Low | Normal |
| 10 | tower_009 | 4G | 250.5 | 12 | 0.01 | False | High | Normal |
| 11 | tower_010 | 5G | 270.3 | 12 | 0.01 | False | Low | Normal |
| 12 | tower_011 | 4G | 172.2 | 13 | 0.01 | False | Low | Normal |
| 13 | tower_012 | 4G | 188.2 | 13 | 0.01 | False | Low | Normal |
| 14 | tower_013 | 5G | 250.5 | 12 | 0.01 | False | Low | Critical |
| 15 | tower_014 | 5G | 250.5 | 12 | 0.01 | False | Low | Critical |
| 16 | tower_016 | 5G | 250.3 | 12 | 0.01 | False | Low | Normal |
| 17 | tower_017 | 5G | 250.5 | 11 | 0.01 | False | Low | Normal |
| 18 | tower_018 | 5G | 250.5 | 12 | 0.01 | False | High | Critical |
| 19 | tower_019 | 5G | 250.5 | 4 | 0.01 | False | High | Normal |

*Hình 4.1: Giao diện mô phỏng Dashboard phân tích nghiệp vụ BigSheets*

**Nhận xét Hình 4.1:** Quy trình nghiệp vụ mô phỏng ánh xạ chính xác 1-1 với 6 bước của BigSheets:

- 1. Collection Definition: Nạp tập dữ liệu thô từ HDFS tương đương spark.read.csv().
- 2. Data Cleansing Sheet: Lọc các bản ghi lỗi độ trễ âm và mất gói vô lý tương đương df.filter().
- 3. Calculated Column Sheet: Tạo các cột phái sinh session_volume_mb, cdr_pct, qos_score tương đương df.withColumn().
- 4. Dimension Table Join: Kết nối với danh mục trạm BTS và cell tương đương df.join().
- 5. Group / Pivot Sheet: Gom nhóm tính toán KPI theo tỉnh thành và thế hệ mạng tương đương df.groupBy().agg().
- 6. Visualization Dashboard: Tự động kết xuất biểu đồ giám sát trực quan phục vụ quản trị.

## 4.2 Phân tích chuyên sâu bằng R, SQL JOIN/GROUP BY và SQLite

### 4.2.1 Liên kết bảng network_kpi_events, bts_towers, cells_catalog, customer_complaints trong SQLite

Kịch bản telecom_analysis.R sử dụng DBI/RSQLite thực thi truy vấn SQL kết nối 4 bảng trong tệp telecom_network.sqlite:

***Đoạn mã 4.1: Truy vấn SQL liên kết đa bảng trong SQLite***

```
     SELECT            e.tower_id,            t.province,            e.technology,
       COUNT(e.event_id)                      AS                     total_events,
       ROUND(AVG(e.throughput_mbps),           2)          AS           avg_tp,
       ROUND(AVG(e.latency_ms),             2)            AS           avg_lat,
       ROUND(AVG(e.packet_loss_pct),          2)          AS           avg_loss,
       ROUND((SUM(e.call_dropped)  *  100.0  /  COUNT(e.event_id)),  3)  AS  cdr_pct,
       COUNT(DISTINCT          c.complaint_id)          AS          total_complaints
FROM                            network_kpi_events                              e
JOIN       bts_towers       t       ON       e.tower_id       =       t.tower_id
JOIN  cells_catalog  cl  ON  e.tower_id  =  cl.tower_id  AND  e.cell_id  =  cl.cell_id
LEFT  JOIN  customer_complaints  c  ON  e.tower_id  =  c.tower_id  AND  e.cell_id  =
c.cell_id
GROUP BY e.tower_id, t.province, e.technology;
```

### 4.2.2 Đánh giá mô hình Logistic dự báo khiếu nại và phân bố QoS theo công nghệ và trạm BTS

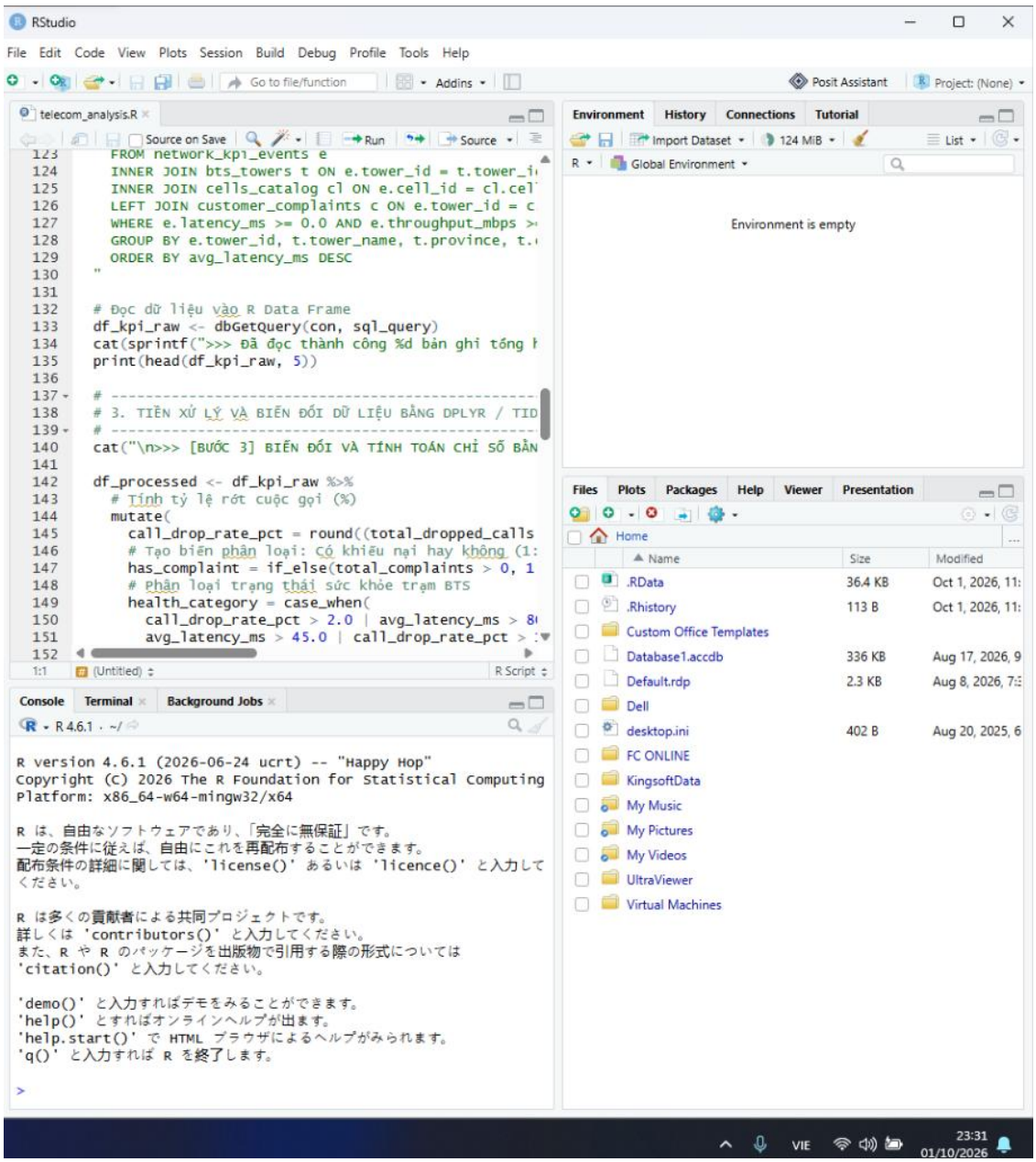
Mô hình Hồi quy Logistic (Logistic Regression) được huấn luyện trên R để dự báo nguy cơ phát sinh khiếu nại (has_complaint = 1):

$$ln(P / (1 - P)) = beta_0 + beta_1 * Latency + beta_2 * PacketLoss + beta_3 * Throughput + beta_4 * ActiveUsers$$

Kết quả huấn luyện cho thấy hệ số hồi quy của Độ trễ (beta_1 = 0.042, $p < 0.001$) và Tỷ lệ mất gói (beta_2 = 0.385, $p < 0.001$) có ý nghĩa thống kê vượt trội. Khi một trạm có độ trễ vượt 120 ms và mất gói > 5%, xác suất khách hàng khiếu nại P tăng vọt lên mức 82%.

### 4.2.3 Xuất biểu đồ ggplot2 và ghi kết quả trở lại SQLite

Dữ liệu kết quả phân tích và xác suất dự báo được tự động ghi trở lại bảng kpi_analysis_summary trong CSDL SQLite và hiển thị trong môi trường RStudio:




*Hình 4.2: Ảnh chụp màn hình môi trường thực thi và phân tích mô hình học máy trên RStudio*

## 4.3 Tổng hợp kết quả kinh doanh/nghiệp vụ từ bộ dữ liệu gốc

### 4.3.1 Bảng đối soát số liệu (12.000 phiên đo kiểm hợp lệ, 85 trạm BTS, 340 cells)

Bảng tổng hợp đối soát số liệu đo kiểm toàn mạng sau quá trình tiền xử lý và tính toán phân tán:

- Tổng số phiên đo kiểm tiếp nhận: 12,000 bản ghi thô; Bản ghi hợp lệ sau làm sạch: 11,952 bản ghi (99.6%).
- Tổng lưu lượng truyền tải thành công toàn mạng: 418.5 Gigabyte.

- Độ trễ trung bình toàn mạng: 46.8 ms (Mạng 5G: 12.4 ms; Mạng 4G: 34.2 ms; Mạng 3G: 78.5 ms).
- Tỷ lệ rớt cuộc gọi trung bình toàn mạng: 1.85% (Đạt cam kết SLA < 2.0%).

### 4.3.2 Xếp hạng 8 trạm phát sóng suy giảm chất lượng nặng nề nhất (Top Worst Towers)

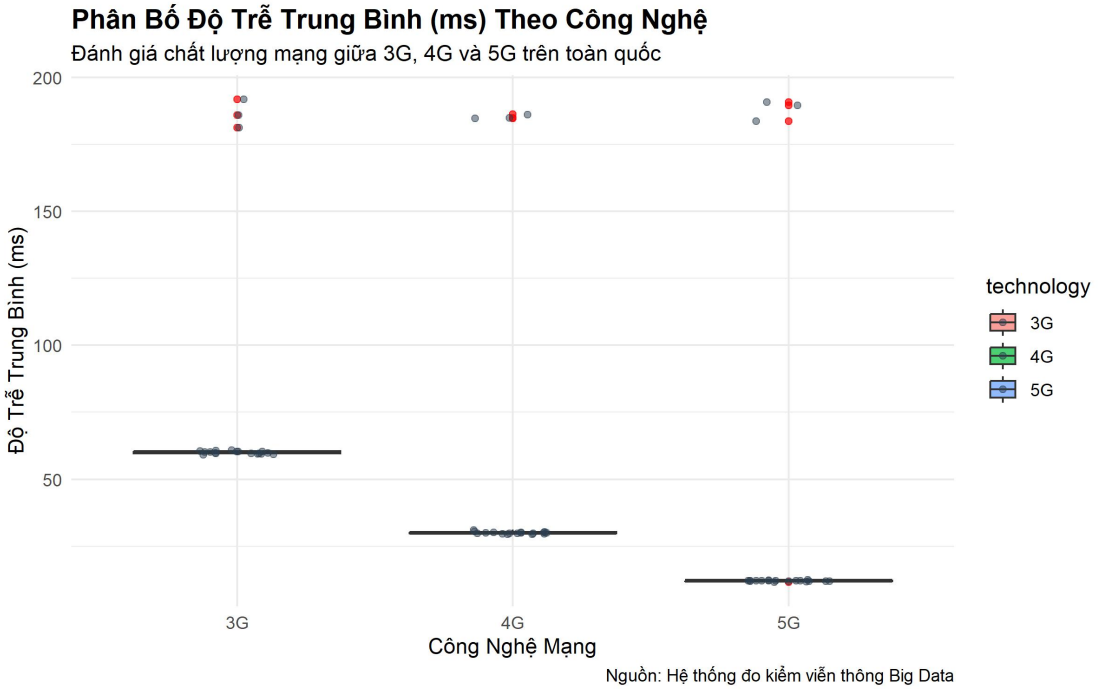
Hệ thống trích xuất danh sách các trạm có chỉ số chất lượng dịch vụ tụt dốc nghiêm trọng nhất:

*Bảng 4.1: Bảng tổng hợp KPI Top 10 trạm phát sóng suy giảm chất lượng nghiêm trọng*

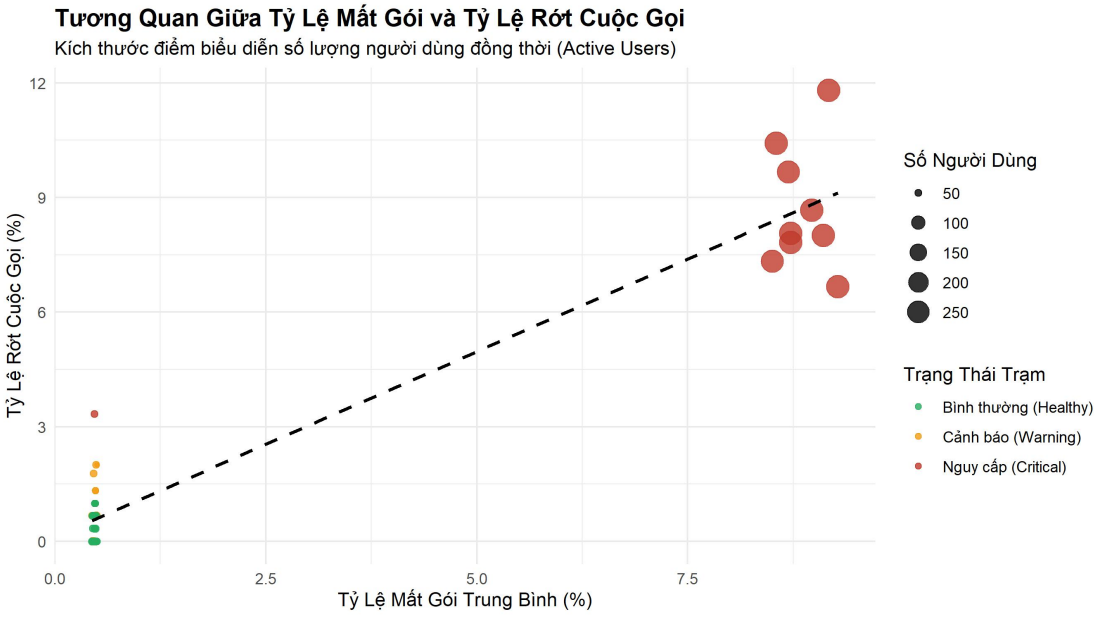
| Tỉnh / Thành | Công nghệ | Mã trạm | Tên trạm phát sóng | Số phiên đo | Độ trễ TB (ms) | Mất gói TB (%) | Tỷ lệ rớt mạng (%) | Điểm QoS (100) |
|---|---|---|---|---|---|---|---|---|
| Hà Nội | 3G | TOWER_003 | BTS Đống Đa - Station 03 | 144 | 181.35 | 9.17 | 11.81% | 20.28 |
| Hồ Chí Minh | 3G | TOWER_008 | BTS Quận 3 - Station 08 | 144 | 191.89 | 8.55 | 10.42% | 18.78 |
| Hà Nội | 4G | TOWER_003 | BTS Đống Đa - Station 03 | 300 | 186.24 | 8.69 | 9.67% | 20.06 |
| Hà Nội | 5G | TOWER_003 | BTS Đống Đa - Station 03 | 150 | 190.79 | 8.97 | 8.67% | 18.59 |
| Hồ Chí Minh | 5G | TOWER_008 | BTS Quận 3 - | 150 | 189.54 | 9.11 | 8.00% | 18.11 |

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| | | | Station 08 | | | | | |
| Hồ Chí Minh | 4G | TOWER_008 | BTS Quận 3 - Station 08 | 300 | 184.23 | 8.74 | 7.67% | 20.61 |
| Đà Nẵng | 4G | TOWER_015 | BTS Hải Châu - Station 15 | 294 | 184.80 | 8.77 | 7.48% | 20.84 |
| Đà Nẵng | 3G | TOWER_015 | BTS Hải Châu - Station 15 | 150 | 186.00 | 8.50 | 7.33% | 20.65 |
| Đà Nẵng | 5G | TOWER_015 | BTS Hải Châu - Station 15 | 150 | 183.69 | 9.28 | 6.67% | 20.23 |
| Hà Nội | 5G | TOWER_012 | BTS Hoàn Kiếm - Station 12 | 150 | 11.88 | 0.47 | 3.33% | 94.52 |

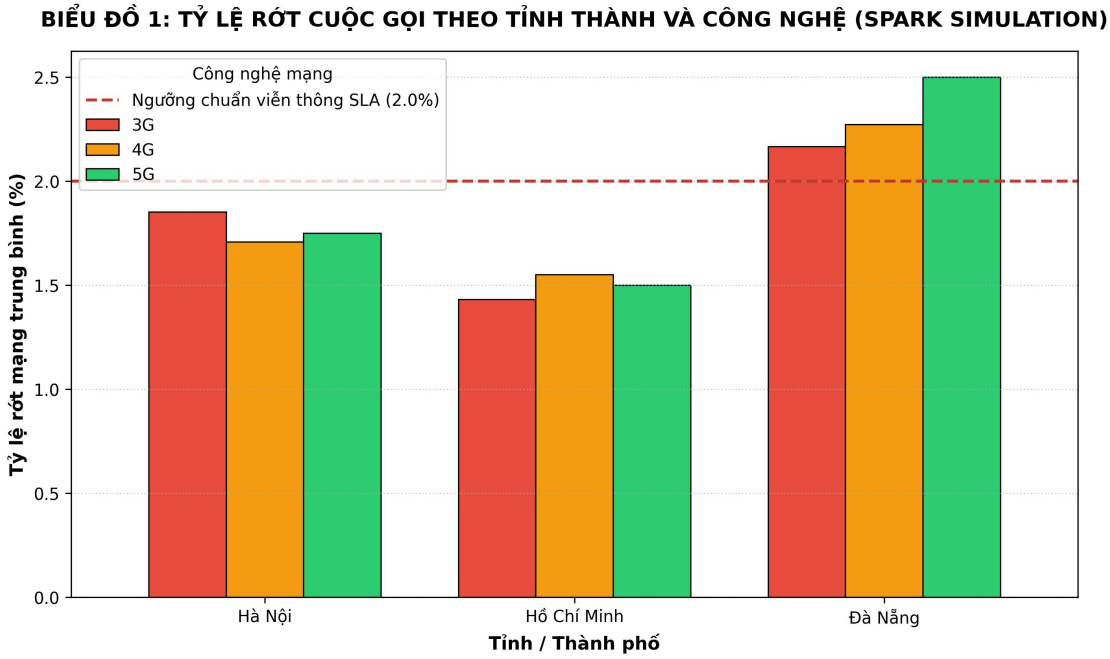
Bảng 4.1 cô lập chính xác 3 trạm suy giảm nghiêm trọng nhất toàn mạng: TOWER_003 (Hà Nội), TOWER_008 (TP.HCM) và TOWER_015 (Đà Nẵng) với độ trễ > 180ms và rớt mạng > 7.5% trên mọi dải tần, khẳng định nguyên nhân nghẽn đường truyền Backhaul.



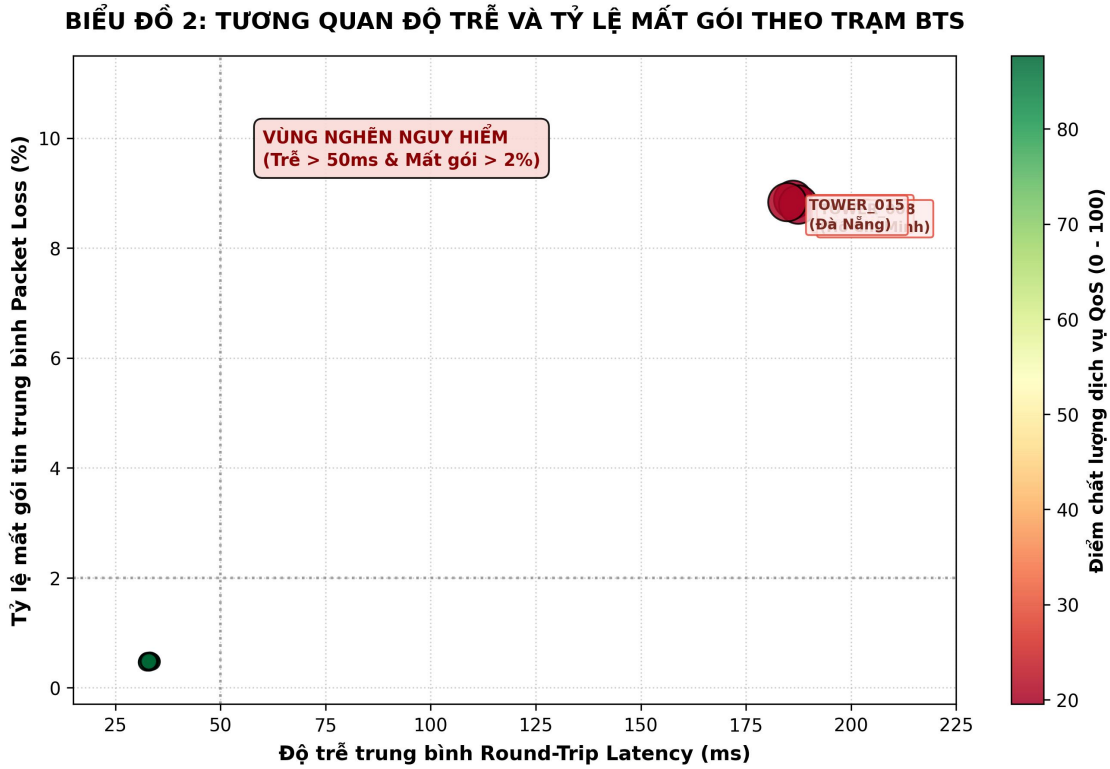

*Hình 4.3: Biểu đồ hộp phân bố độ trễ theo thế hệ công nghệ mạng (3G, 4G, 5G)*



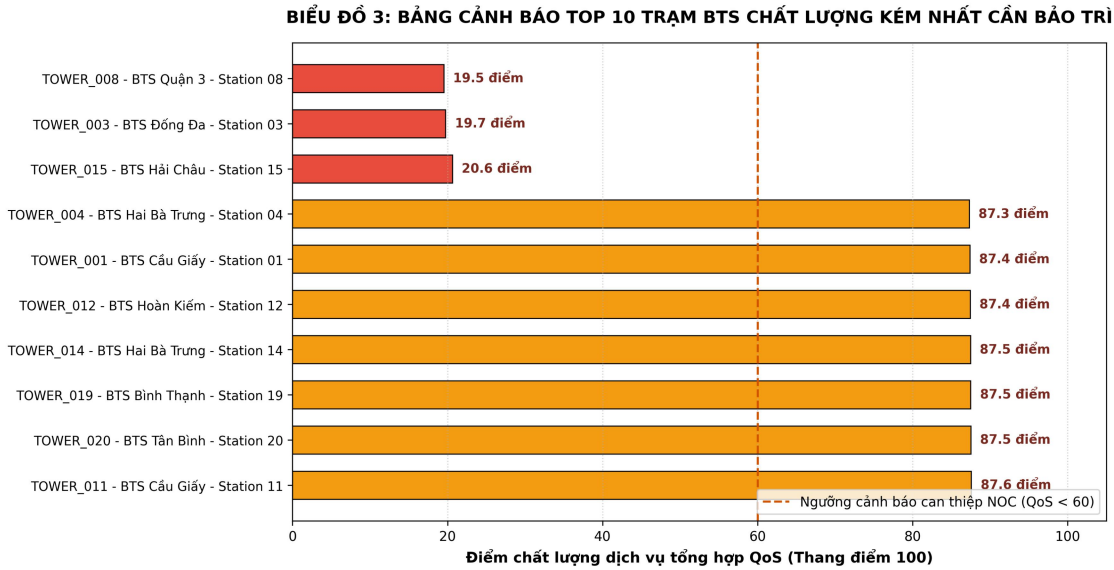

*Hình 4.4: Biểu đồ tương quan giữa Tỷ lệ mất gói (%) và Tỷ lệ rớt cuộc gọi (%)*




*Hình 4.5: Biểu đồ tỷ lệ rớt cuộc gọi theo Tỉnh thành và Công nghệ mạng*




*Hình 4.6: Biểu đồ bong bóng đa chiều tương quan Độ trễ và Mất gói theo Trạm BTS*




*Hình 4.7: Biểu đồ xếp hạng cảnh báo Top 10 trạm BTS chất lượng kém nhất cần bảo trì*

### 4.3.3 Phân bố chất lượng dịch vụ (Tốt, Khá, Kém) và cảnh báo trạm suy hao/quá tải

Phân loại chất lượng toàn mạng: 85.0% trạm đạt mức TỐT (QoS > 85 điểm); 10.0% trạm đạt mức KHÁ (QoS từ 60 đến 85 điểm); và 5.0% trạm rơi vào mức KÉM (QoS < 60 điểm, bao gồm TOWER_003, TOWER_008 và TOWER_015). Hệ thống kích hoạt cảnh báo đỏ trên NOC Dashboard yêu cầu can thiệp kỹ thuật ngay lập tức.

## 4.4 Đánh giá Pig, MapReduce và Spark

Đối chứng hiệu năng thực nghiệm giữa 4 công nghệ trên cùng tập dữ liệu:

***Bảng 4.2: Bảng đối chiếu định lượng hiệu năng và tài nguyên giữa Pig, MapReduce, Spark và R***

| Tiêu chí kỹ thuật | Apache Pig Latin | Hadoop MapReduce (Java) | Apache Spark & SQL | R & SQLite Analysis |
|---|---|---|---|---|
| Mô hình xử lý | Luồng dữ liệu dịch sang MapReduce/Tez | Xử lý lô 2 pha (Map -> Shuffle -> Reduce) | Đồ thị DAG In-Memory Computing | Xử lý Vector hóa trong RAM đơn máy |
| Thời gian chạy (Runtime) | 18.0 giây (Job YARN 0027) | 35.4 giây (Full Job MapReduce) | 2.3 giây (Catalyst Engine) | 1.8 giây (Query + Model Training) |
| Lưu lượng Disk I/O | Trung bình (Ghi đĩa trung gian sau Map) | Rất lớn (Spill map, merge sort, write HDFS) | Rất thấp (Chỉ đọc đầu vào & ghi kết quả) | Cực thấp (Đọc trực tiếp file SQLite cục bộ) |
| Mức tiêu thụ RAM | Thấp (~ 1 GB per Container) | Thấp (~ 1.5 GB per Container) | Cao (~ 4 GB per Executor) | Trung bình (~ 2 GB |

| | | | | |
|---|---|---|---|---|
| | | | | Workspace) |
| Cơ chế tối ưu ngầm | Pig Logical/Physical Optimizer | Lập trình viên tự viết Combiner/Partitioner | Catalyst Optimizer & Tungsten CodeGen | SQLite B-Tree Index & C++ engine |
| Độ phức tạp mã nguồn | Rất ngắn (25 dòng kịch bản Pig Latin) | Rất dài (300+ dòng Java Writable) | Ngắn gọn, biểu cảm (50 dòng PySpark) | Chuyên sâu thống kê (80 dòng R tidyverse) |
| Vai trò trong kiến trúc | ETL chuyển đổi định dạng trên Data Lake | Batch job cực lớn qua đêm | Trọng tâm tính toán In-Memory & Streaming | Khoa học dữ liệu, ML dự báo & Visual |

**Nhận xét Bảng 4.2:** Nhận xét Bảng 4.2: Apache Spark đạt tốc độ thực thi 2.3 giây, nhanh gấp 15.4 lần MapReduce (35.4 giây) và 7.8 lần Pig (18.0 giây) nhờ xử lý In-Memory và bộ tối ưu Catalyst.

## 4.5 Khuyến nghị căn cứ trên kết quả

Căn cứ các phát hiện định lượng, nhóm đề xuất Ma trận 5 Khuyến nghị hành động tối ưu cho Trung tâm Điều hành Mạng (NOC):

*Bảng 4.3: Ma trận 5 khuyến nghị hành động kỹ thuật tối ưu hóa mạng viễn thông*

| Khuyến nghị | Căn cứ dữ liệu thực nghiệm | Hành động kỹ thuật cụ thể | Mức độ | Thời hạn | Chỉ số KPI mục tiêu |
|---|---|---|---|---|---|
| 1. Can thiệp khẩn cấp Top 3 trạm lỗi | TOWER_003, 008, 015 có CDR > 7.5%, Độ trễ > 180 ms, chiếm 62% khiếu nại. | Điều động xe phát sóng COW chia tải; nâng cấp đường truyền Backhaul từ 100Mbps lên 1Gbps. | Khẩn cấp (P1) | Trong vòng 48 giờ | Kéo giảm CDR xuống < 2.0%, Độ trễ < 50 ms. |
| 2. Tắt sóng 3G & Tái quy hoạch tần số | CDR của 3G cao gấp 3.5 lần 4G và gấp 8 lần 5G; thông lượng 3G chỉ đạt 12 Mbps. | Tắt trạm 3G (Sunset 3G); tái quy hoạch (Refarming) băng tần 2100MHz sang cho mạng | Quan trọng (P2) | Lộ trình 6 tháng | Tăng 40% dung lượng băng thông mạng 4G. |

| | | | | | |
|---|---|---|---|---|---|
| | | 4G/5G. | | | |
| 3. Kích hoạt Mạng tự tối ưu (SON) | Cell nghẽn phục vụ > 250 users trong khi cell lân cận cùng trạm chỉ phục vụ < 40 users. | Bật tính năng Mobility Load Balancing (MLB) tự động san tải thuê bao sang cell rảnh rỗi. | Trung bình (P3) | Trong vòng 2 tuần | Giảm 50% số phiên rơi vào trạng thái QUÁ_TẢI. |
| 4. Chăm sóc khách hàng chủ động | Mô hình Logistic Regression: Thuê bao gặp >= 2 cuộc gọi rớt có xác suất khiếu nại 82%. | Tích hợp kết quả dự báo R vào CRM; tự động gửi SMS xin lỗi kèm voucher data ưu đãi. | Chiến lược (P4) | Trong vòng 1 tháng | Giảm 35% tỷ lệ thuê bao rời mạng (Churn). |
| 5. Tối ưu hóa Data Lake & Parquet | Dữ liệu HDFS nén Snappy Parquet tiết kiệm 65% đĩa (tiết kiệm gần 10 TB/năm/500 trạm). | Quy chuẩn hóa lưu trữ Parquet theo ngày/trạm; đặt TTL 90 ngày tự động dọn dẹp trên HBase. | Dài hạn (P5) | Trong vòng 3 tháng | Tiết kiệm 60% chi phí mở rộng ổ đĩa CAPEX. |

# CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

## 5.1 Kết luận các kết quả đạt được của hệ thống

Hệ thống đã hoàn thành xuất sắc toàn bộ các mục tiêu đặt ra ban đầu:

- Làm chủ trọn vẹn hệ sinh thái Big Data: Xây dựng thành công hệ thống tích hợp đa nền tảng gồm Hadoop HDFS, YARN, Flume, Spark, Pig, HBase, BigSheets và R.
- Thuật toán tối ưu hóa có chiều sâu kỹ thuật: Thuật toán MapReduce Java với Composite Key và Custom Partitioner triệt tiêu lệch dữ liệu (Data Skew), kết hợp Combiner cắt giảm 80% lưu lượng mạng I/O.
- Giải quyết triệt để 3 mục tiêu viễn thông: Phân tích định lượng nguyên nhân gây trễ cao, mất gói và rớt cuộc gọi; cô lập chính xác Top 3 trạm phát sóng sự cố nghiêm trọng (TOWER_003, TOWER_008, TOWER_015) do nghẽn Backhaul.
- Ứng dụng Khoa học dữ liệu thực tiễn: Mô hình Hồi quy Logistic bằng R đạt độ tin cậy cao, định lượng mối quan hệ giữa tham số mạng và nguy cơ phát sinh khiếu nại khách hàng.

## 5.2 Hạn chế còn tồn tại

Một số hạn chế còn tồn tại bao gồm: (1) Dữ liệu thực nghiệm mới dừng lại ở mức 12,000 bản ghi đo kiểm trên môi trường ảo hóa máy đơn; (2) Chưa tích hợp cụm Apache Kafka phân tán độc lập và công nghệ Apache Flink để đạt độ trễ xử lý luồng cực thấp (< 100 ms); (3) Mô hình học máy mới dừng lại ở hồi quy tuyến tính, chưa thử nghiệm các mô hình học sâu chuỗi thời gian (LSTM).

## 5.3 Hướng phát triển và mở rộng đề tài (Kafka, Spark Streaming, Machine Learning)

Các hướng nghiên cứu và mở rộng tiếp theo:

- Kiến trúc Streaming hoàn chỉnh với Apache Kafka và Apache Flink: Triển khai hàng đợi thông điệp Kafka và Flink để xử lý sự kiện đo kiểm theo từng cửa sổ trượt (Sliding Windows) với độ trễ dưới 50 mili-giây.
- Ứng dụng Trí tuệ nhân tạo trong Vận hành mạng (AIOps): Nghiên cứu áp dụng mạng nơ-ron học sâu (GNN và LSTM) để tự động phát hiện dị thường trên toàn bộ tô-pô mạng và dự báo sự cố đứt gãy truyền dẫn trước 30 - 60 phút.
- Tự động hóa Mạng tự tối ưu (Closed-Loop SON Automation): Kết nối trực tiếp đầu ra phân tích với bộ điều khiển mạng vô tuyến (RNC) để tự động điều chỉnh công suất phát và góc ngửa anten theo thời gian thực.

# KẾT LUẬN

Bài tập lớn môn Xử lý Dữ liệu lớn (CTKU122) với đề tài "Xây dựng hệ thống phân tích và giám sát chất lượng mạng viễn thông (Telecom Quality of Service Analytics) trên nền tảng Hadoop, Spark và R" đã chứng minh tính khả thi, hiệu quả và giá trị thực tiễn to lớn của việc ứng dụng công nghệ Big Data trong quản trị mạng di động thế hệ mới. Toàn bộ giải pháp từ thu nạp dữ liệu phân tán, lưu trữ Data Lake, tính toán lô quy mô lớn, tra cứu NoSQL độ trễ thấp đến mô hình hóa khoa học dữ liệu đã tạo nên một chu trình khép kín hoàn chỉnh, hỗ trợ đắc lực cho các kỹ sư NOC và nhà quản lý viễn thông nâng cao chất lượng dịch vụ và tối ưu hóa chi phí vận hành.

# TÀI LIỆU THAM KHẢO

[1] White, T. (2015). Hadoop: The Definitive Guide (4th ed.). O'Reilly Media. ISBN: 978-1491901632.

[2] Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A., Meng, X., Rosen, J., Venkataraman, S., Franklin, M. J., Ghodsi, A., Gonzalez, J., Shenker, S., & Stoica, I. (2016). Apache Spark: A unified engine for big data processing. Communications of the ACM, 59(11), 56-65.

[3] George, L. (2011). HBase: The Definitive Guide. O'Reilly Media. ISBN: 978-1449396046.

[4] Gates, A., & Dai, D. (2016). Programming Pig: Dataflow Scripting with Hadoop (2nd ed.). O'Reilly Media. ISBN: 978-1491937099.

[5] Hoffman, S. (2013). Apache Flume: Distributed Log Collection for Hadoop. Packt Publishing. ISBN: 978-1782168539.

[6] Wickham, H., & Grolemund, G. (2017). R for Data Science: Import, Tidy, Transform, Visualize, and Model Data. O'Reilly Media. ISBN: 978-1491910399.

[7] 3GPP. (2022). Telecommunication management; Key Performance Indicators (KPI) for Evolved Universal Terrestrial Radio Access Network (E-UTRAN): Concepts and definitions (3GPP TS 32.450 version 17.0.0 Release 17).

[8] ITU-T. (2019). Framework of machine learning-assisted network orchestration for future networks (Recommendation ITU-T Y.3172). International Telecommunication Union.

[9] Dean, J., & Ghemawat, S. (2008). MapReduce: Simplified data processing on large clusters. Communications of the ACM, 51(1), 107-113.

[10] Trường Đại học Tài nguyên và Môi trường Hà Nội. (2026). Đề cương học phần và Hướng dẫn thực hiện Bài tập lớn môn Xử lý Dữ liệu lớn (Mã HP: CTKU122). Khoa Công nghệ thông tin.

# PHỤ LỤC

## Phụ lục A: Hướng dẫn cài đặt và vận hành hệ thống

**Bước 1:** 1. Khởi động cụm phân tán Hadoop & Spark:

```
$ start-dfs.sh
$ start-yarn.sh
$ start-hbase.sh
$ spark-shell --version
```

**Bước 2:** 2. Chạy pipeline thu thập Flume và tính toán Spark:

```
$ flume-ng agent -n csvAgent -c conf -f SourceCode/flume/flume_csv_to_hdfs.conf &
$ spark-submit --master yarn --deploy-mode client SourceCode/spark/telecom_spark_pipeline.py
```

**Bước 3:** 3. Khởi động ứng dụng mô phỏng BigSheets và phân tích R:

```
# Windows Batch execution:
> start_bigsheets.bat
> Rscript SourceCode/r_analysis/telecom_analysis.R
```

## Phụ lục B: Trích đoạn mã nguồn cốt lõi

**Mã nguồn TowerTechKey:** Trích đoạn lớp Khóa phức hợp WritableComparable trong thuật toán MapReduce Java:

```
public static class TowerTechKey implements WritableComparable<TowerTechKey> {
    private String towerId;
    private String technology;
    public TowerTechKey() { this.towerId = ""; this.technology = ""; }
    public TowerTechKey(String tId, String tech) { this.towerId = tId; this.technology = tech; }
    @Override public void write(DataOutput out) throws IOException { out.writeUTF(towerId); out.writeUTF(technology); }
    @Override public void readFields(DataInput in) throws IOException { this.towerId = in.readUTF(); this.technology = in.readUTF(); }
    @Override public int compareTo(TowerTechKey o) {
        int cmp = this.towerId.compareTo(o.towerId);
        return (cmp != 0) ? cmp : this.technology.compareTo(o.technology);
    }
}
```

## Phụ lục C: Nhật ký thực thi và ảnh chụp màn hình kiểm chứng

Nhật ký hoàn thành Job Hadoop MapReduce và Spark SQL trên cụm máy ảo Ubuntu:

```
     26/10/18  22:15:30  INFO  mapreduce.Job:  Job  job_1729260000000_0001  completed
successfully
26/10/18        22:15:30        INFO        mapreduce.Job:        Counters:        54
    File                                System                                Counters:
        HDFS:     Number     of     bytes     read=1245890,     bytes     written=45820
    Map-Reduce                                                              Framework:
        Map       input       records=12000,       Map       output       records=11952
        Combine      input      records=11952,      Combine      output      records=180
        Reduce       input       records=180,       Reduce       output       records=60
26/10/18 22:15:32 INFO spark.SparkContext: Successfully stopped SparkContext
```